Process Management

Dr Nishant Jain

Assistant Professor

MITS-DU Gwalior MP

Process

A process is a program in execution.

A process is a running program that serves as the foundation for all computation

For example, when we write a program in C or C++ and compile it, the compiler creates binary code. The original code and binary code are both programs. When we actually run the binary code, it becomes a process.

Process vs Program

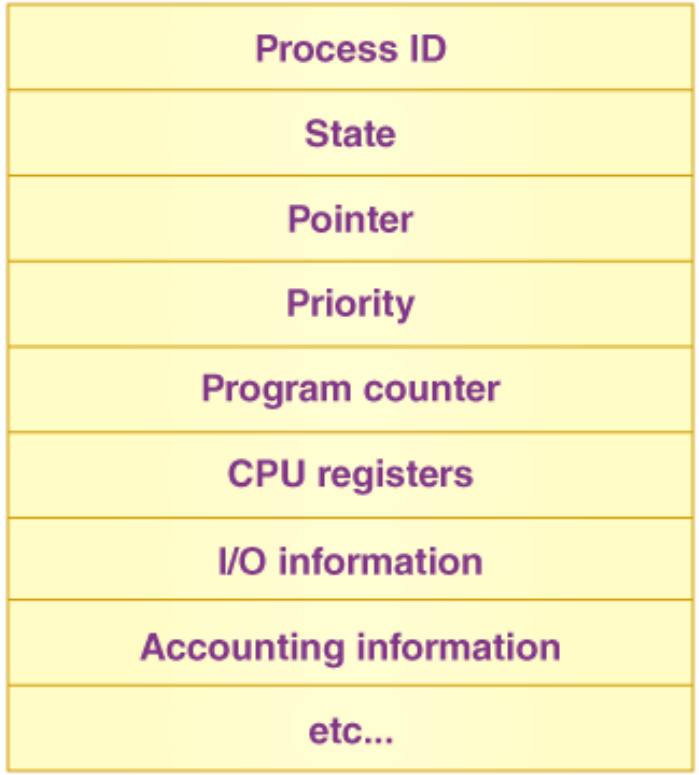
A process is an ‘active’ entity instead of a program, which is considered a ‘passive’ entity. A single program can create many processes when run multiple times; for example, when we open a .exe or binary file multiple times, multiple instances begin (multiple processes are created).

Components of a Process

- **Stack : Temporary data like method or function parameters, return address, and local variables are stored in the process stack.**
- **Heap : This is the memory that is dynamically allocated to a process during its execution.**
- **Text : This comprises the contents present in the processor's registers as well as the current activity reflected by the value of the program counter.**
- **Data : The global as well as static variables are included in this section**

Process Control Block

- **The PCB architecture is fully dependent on the operating system, and different operating systems may include different information. A simplified diagram of a PCB is shown.**
- **The PCB is kept for the duration of a procedure and then removed once the process is finished.**



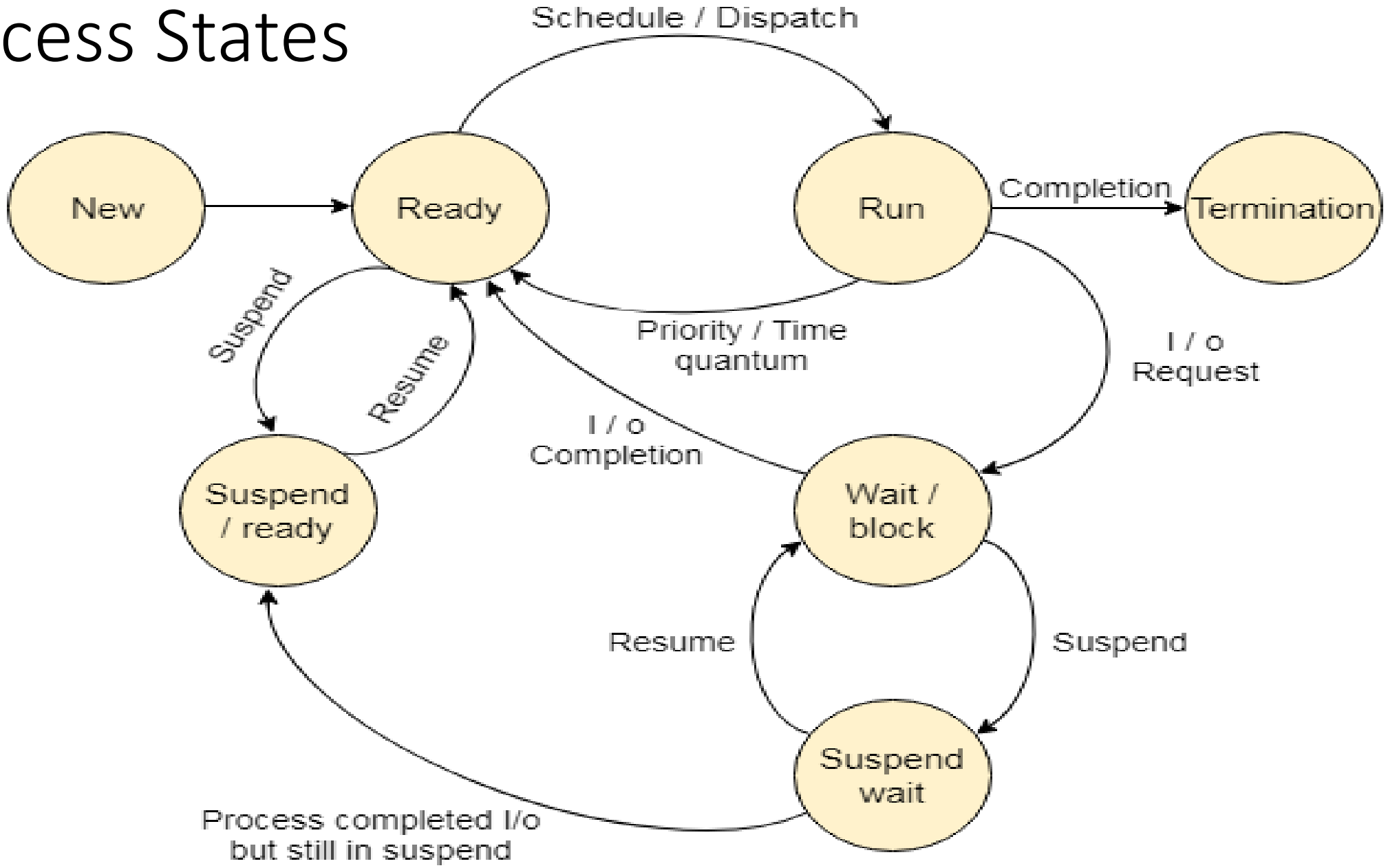
Process Attributes

- **Process state : The process's present state, such as whether it's ready, waiting, running, or whatever.**
- **Process privileges : This is required in order to grant or deny access to system resources.**
- **Process ID : Each process in the OS has its own unique identifier.**
- **Pointer : It refers to a pointer that points to the parent process.**
- **Program counter : The program counter refers to a pointer that points to the address of the process's next instruction.**
- **Priority : Every process has its own priority. The process with the highest priority among the processes gets the CPU first. This is also stored on the process control block.**

Process Attributes

- **CPU registers : Processes must be stored in various CPU registers for execution in the running state.**
- **CPU scheduling information : Process priority and additional scheduling information are required for the process to be scheduled.**
- **Memory management information : This includes information from the page table, memory limitations, and segment table, all of which are dependent on the amount of memory used by the OS.**
- **Accounting information : This comprises CPU use for process execution, time constraints, and execution ID, among other things.**
- **IO status information : This section includes a list of the process's I/O devices.**

Process States



Process States

The process, from its creation to completion, passes through various states. The minimum number of states is five. The names of the states are not standardized although the process may be in one of the following states during execution.

1. New : A program which is going to be picked up by the OS into the main memory is called a new process.

2. Ready : Whenever a process is created, it directly enters in the ready state, in which, it waits for the CPU to be assigned. The processes which are ready for the execution and reside in the main memory are called ready state processes. There can be many processes present in the ready state.

Process States

3. Running : One of the processes from the ready state will be chosen by the OS depending upon the scheduling algorithm

4. Block or wait : From the Running state, a process can make the transition to the block or wait state depending upon the scheduling algorithm or the intrinsic behaviour of the process.

When a process waits for a certain resource to be assigned or for the input from the user then the OS move this process to the block or wait state and assigns the CPU to the other processes.

5. Completion or termination : When a process finishes its execution, it comes in the termination state. All the context of the process (Process Control Block) will also be deleted the process will be terminated by the Operating system.

Process States

6. Suspend ready : A process in the ready state, which is moved to secondary memory from the main memory due to lack of the resources (mainly primary memory) is called in the suspend ready state.

If the main memory is full and a higher priority process comes for the execution then the OS have to make the room for the process in the main memory by throwing the lower priority process out into the secondary memory. The suspend ready processes remain in the secondary memory until the main memory gets available.

7. Suspend wait : Instead of removing the process from the ready queue, it's better to remove the blocked process which is waiting for some resources in the main memory.

Since it is already waiting for some resource to get available hence it is better if it waits in the secondary memory and make room for the higher priority process. These processes complete their execution once the main memory gets available and their wait is finished.

Inter Process Communication

A system can have two types of processes i.e. independent and cooperating. Cooperating processes affect each other and may share data and information among themselves.

“Interprocess Communication or IPC provides a mechanism to exchange data and information across multiple processes, which might be on single or multiple computers connected by a network”.

OR

"Inter-process communication is used for exchanging useful information between numerous threads in one or more processes (or programs)."

Why IPC is required?

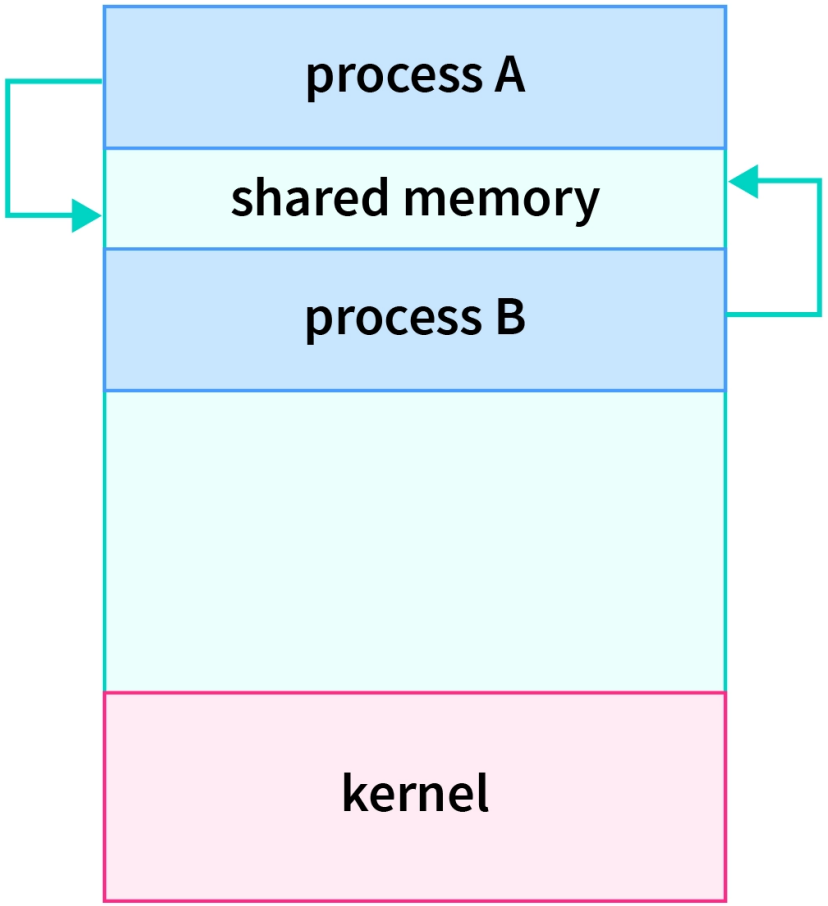


IPC helps achieve these things:

- **Computational Speedup**
- **Modularity**
- **Information and data sharing**
- **Privilege separation**
- **Processes communication and synchronization.**

Shared Memory

Multiple processes can access a common shared memory. Multiple processes communicate by shared memory, where one process makes changes at a time and then others view the change. Shared memory does not use kernel.



Role of Synchronization in IPC

Since multiple processes can read and write on shared variables and data, it is very important to maintain consistency of data, else we can get error-prone results. To prevent this we synchronize the processes. Some common ways to achieve this are given below.

Semaphore: A variable that manages several processes' access to a shared resource. Binary and counting semaphores are the two types of semaphores.

Mutual Exclusion or mutex: is a term used to describe a situation where only one process or thread at a time can enter the crucial part due to mutual exclusion. This avoids race conditions.

Process Scheduling

The act of determining which process is in the ready state and should be moved to the running state is known as Process Scheduling.

The prime aim of the process scheduling system is to keep the CPU busy all the time and to deliver minimum response time for all programs. For achieving this, the scheduler must apply appropriate rules for swapping processes IN and OUT of CPU.

Process Scheduling

Scheduling fell into one of the two general categories:

Non Pre-emptive Scheduling: When the currently executing process gives up the CPU voluntarily.

Pre-emptive Scheduling: When the operating system decides to favour another process, pre-empting the currently executing process.

Scheduling Queue

All processes, upon entering into the system, are stored in the **Job Queue**.

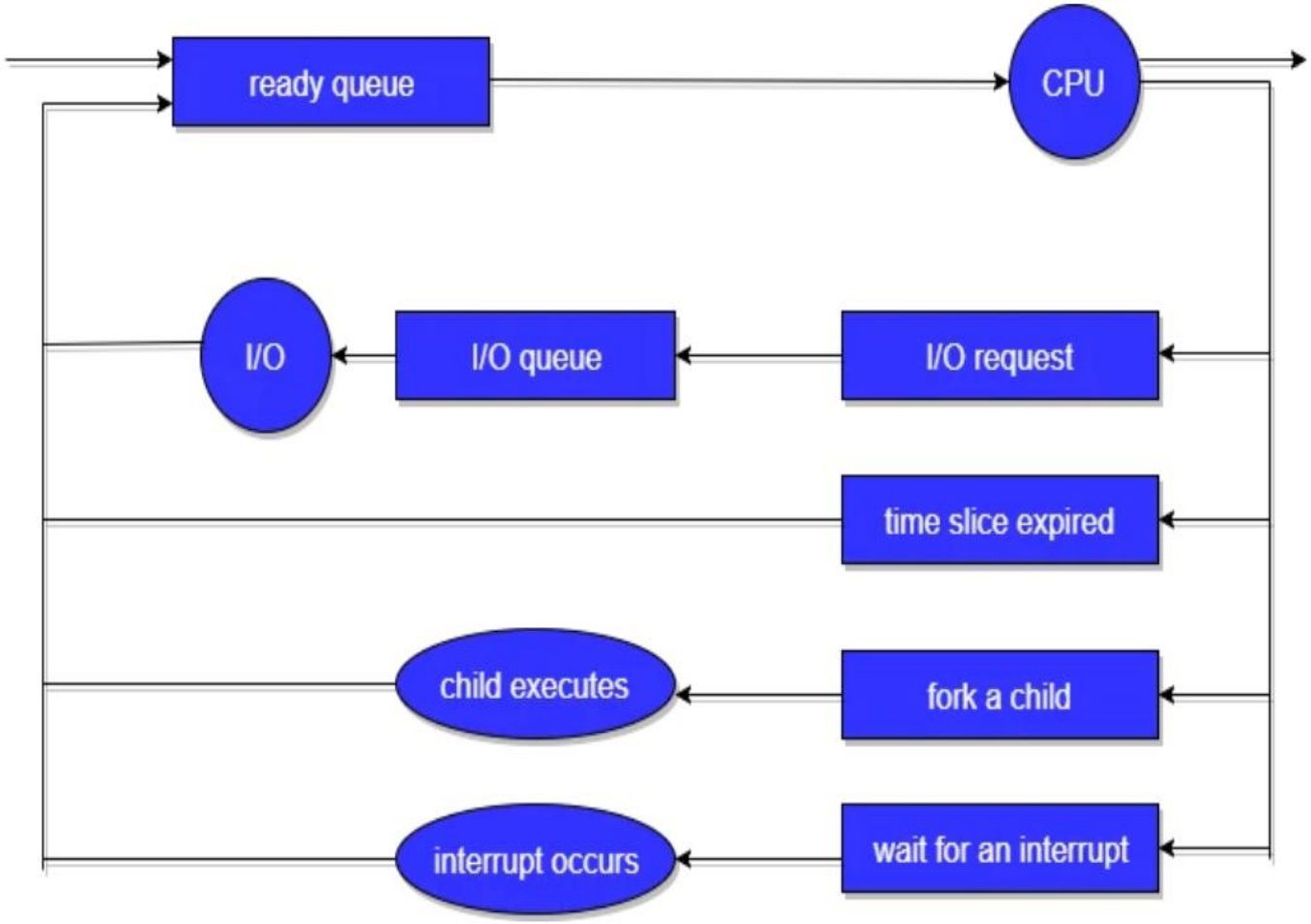
A new process is initially put in the **Ready queue**. It waits in the ready queue until it is selected for execution(or dispatched). Once the process is assigned to the CPU and is executing, one of the following several events can occur:

- The process could issue an I/O request, and then be placed in the **I/O queue (Device Queue)**.
- The process could create a new subprocess and wait for its termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.

Scheduling Queue

In the first two cases, the process eventually switches from the waiting state to the ready state and is then put back in the ready queue.

A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.



Types of Scheduler

There are three types of schedulers:

1. Long Term Scheduler

2. Short Term Scheduler

3. Medium Term Scheduler

Long Term Scheduler

- **It is also called a job scheduler.**
- **It determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution.**
- **The primary objective of the job scheduler is to provide a balanced mix of jobs, such as I/O bound and processor bound.**
- **It also controls the degree of multiprogramming. If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system.**
- **Time-sharing operating systems have no long term scheduler. When a process changes the state from new to ready, then there is use of long-term scheduler.**

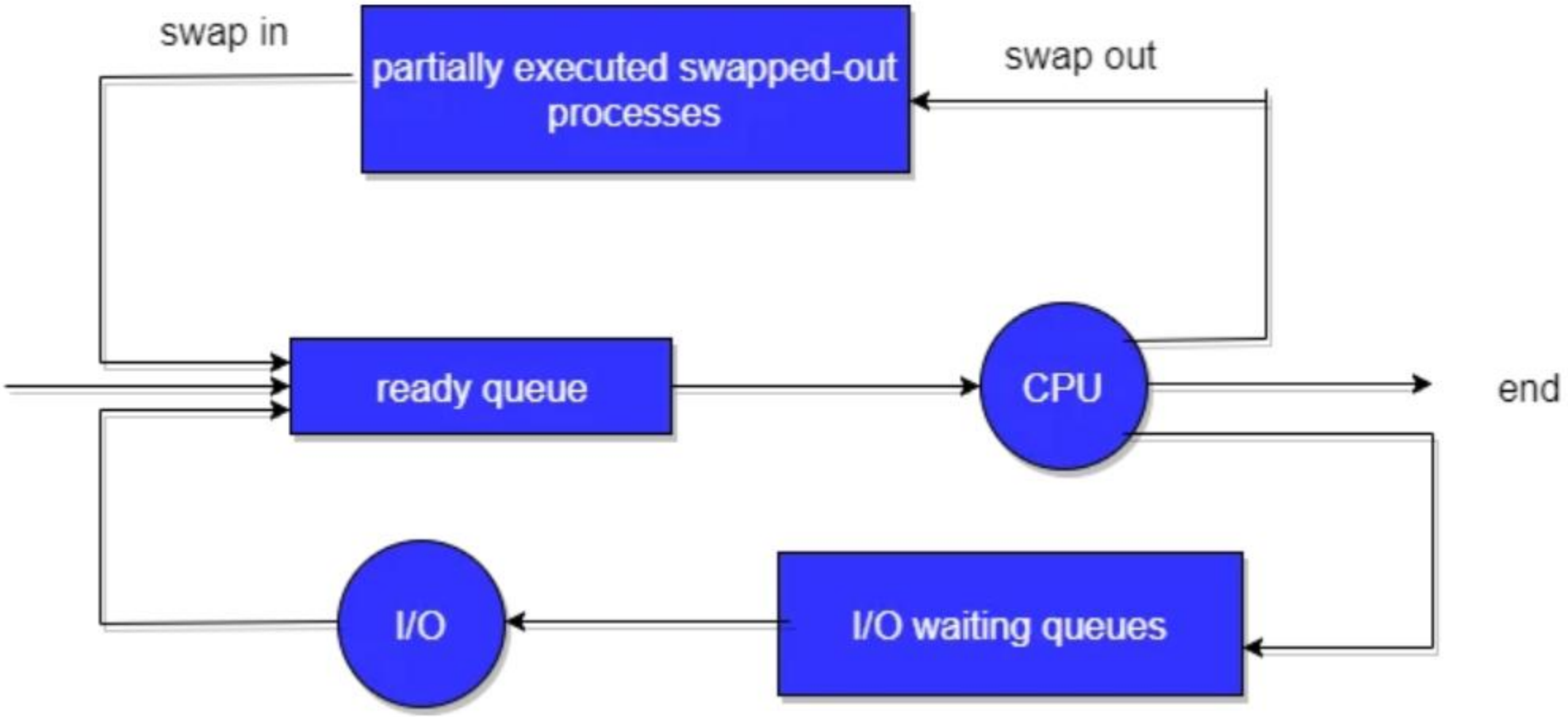
Short Term Scheduler

- **It is also called as CPU scheduler.**
- **Its main objective is to increase system performance in accordance with the chosen set of criteria.**
- **It is the change of ready state to running state of the process.**
- **CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.**
- **Short-term schedulers, also known as dispatchers, make the decision of which process to execute next.**
- **Short-term schedulers are faster than long-term schedulers.**

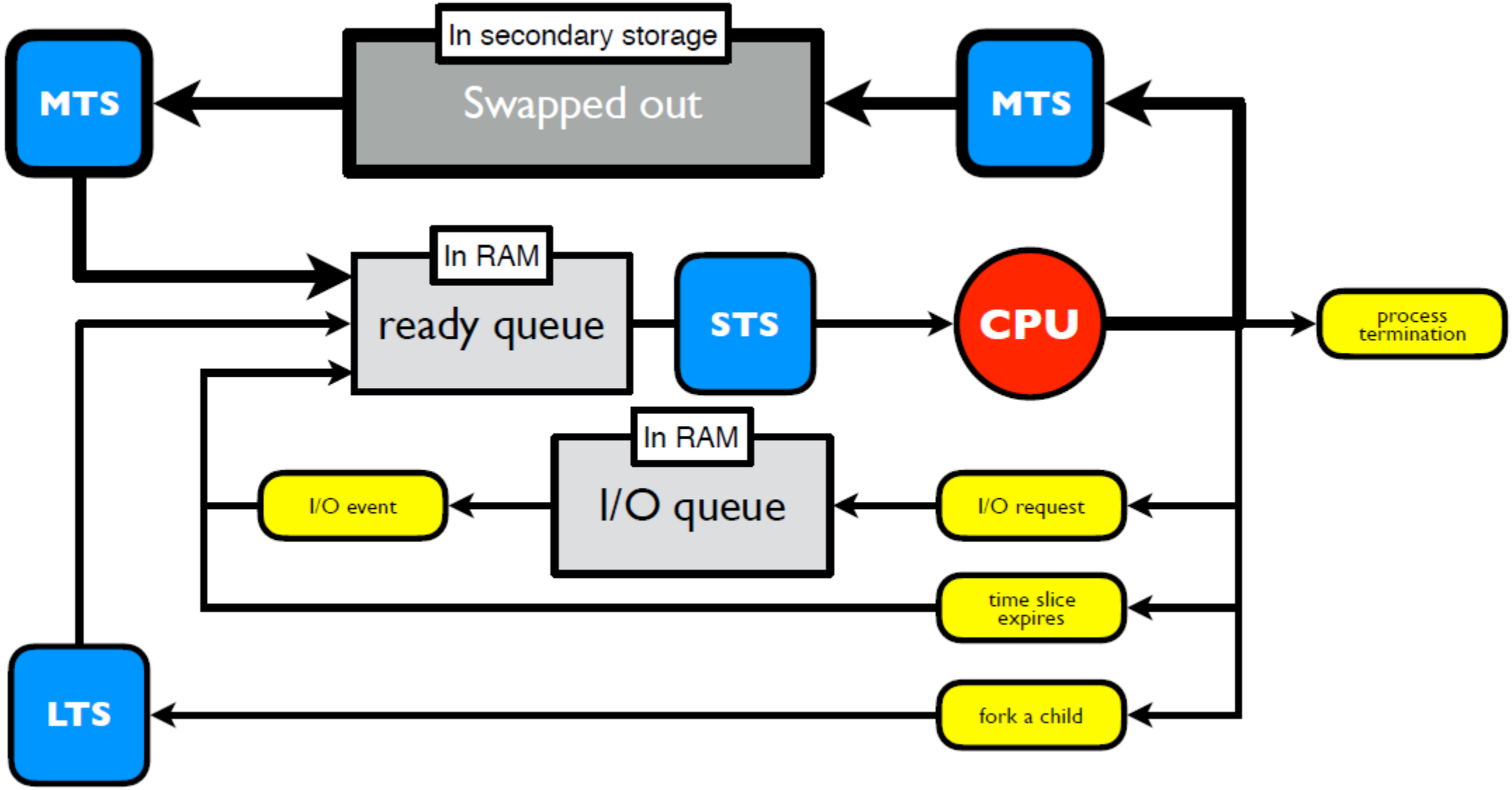
Medium Term Scheduler

- **This scheduler removes the processes from memory (and from active contention for the CPU), and thus reduces the degree of multiprogramming.**
- **At some later time, the process can be reintroduced into memory and its execution can be continued where it left off. This scheme is called swapping. The process is swapped out, and is later swapped in, by the medium term scheduler.**
- **Swapping may be necessary to improve the process mix, or because a change in memory requirements has overcommitted available memory, requiring memory to be freed up.**

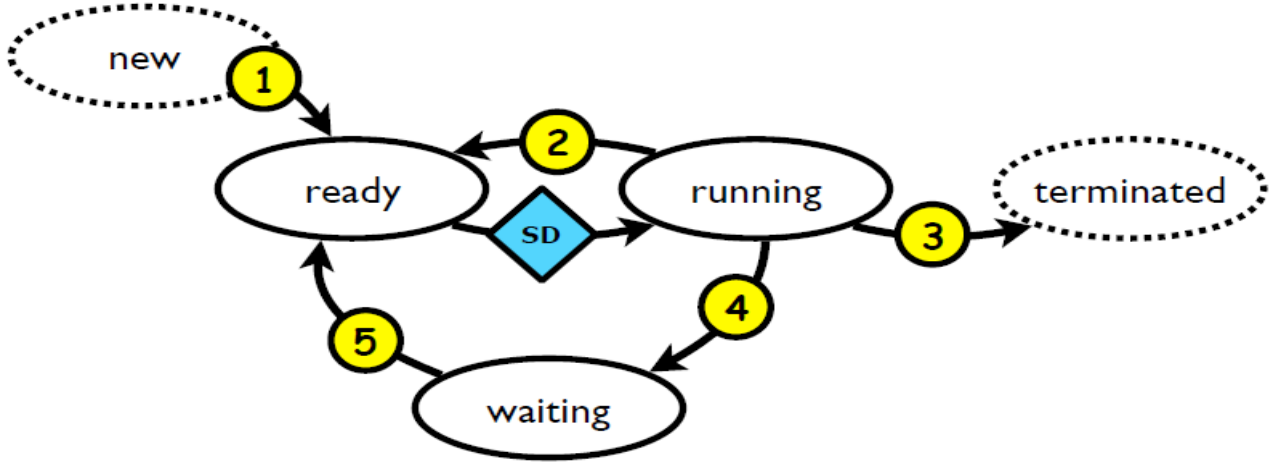
Medium Term Scheduler



Scheduler's


In secondary storage
Swapped out
MTS
MTS
In RAM
ready queue
STS
CPU
process termination
In RAM
I/O queue
I/O event
I/O request
time slice expires
fork a child
LTS

Event Causing Scheduler Dispatch



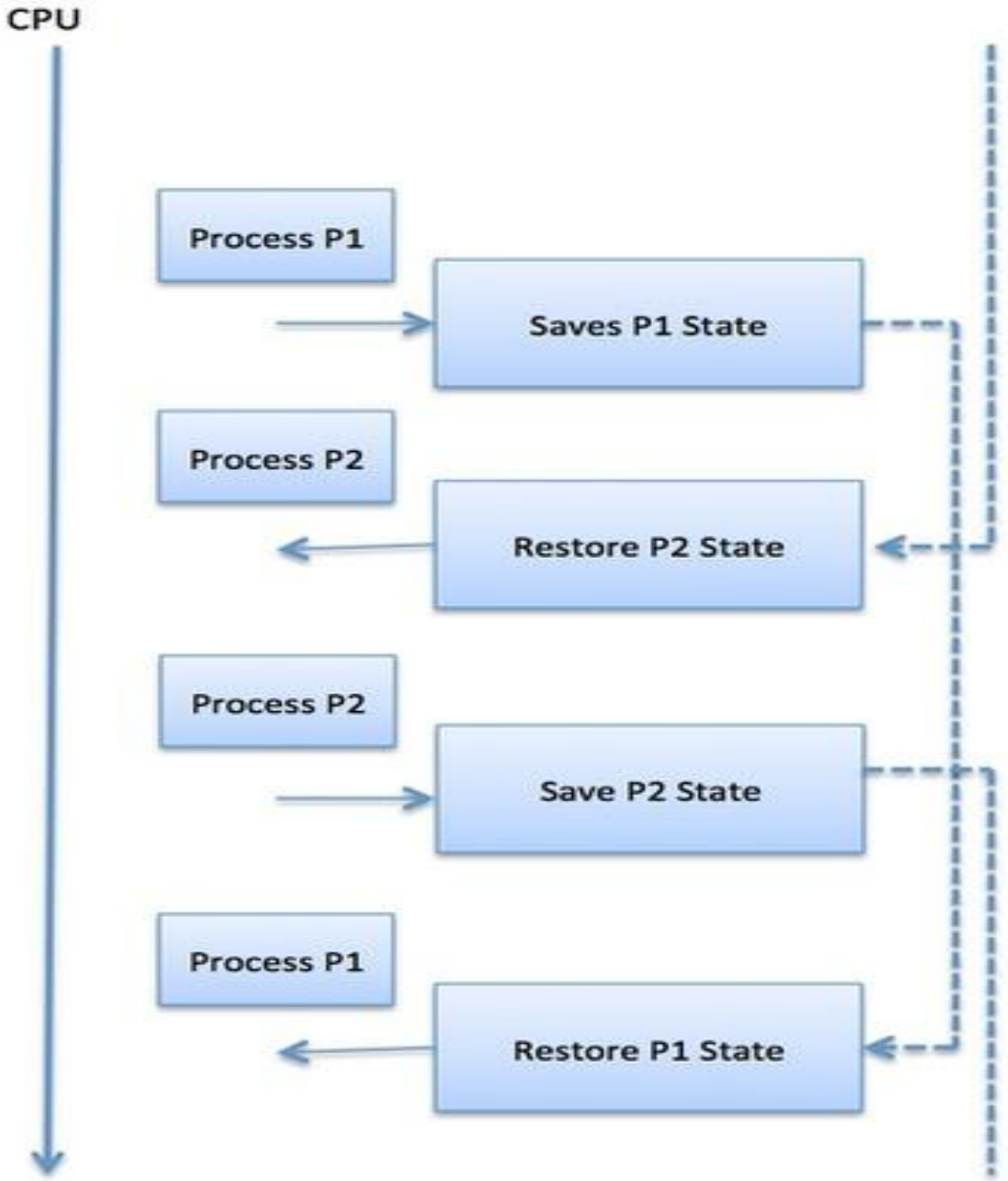
1. A new process arrives at the ready queue.
2. Time slice expires or other forms of **preemption**.
3. The running process terminates.
4. The running process requests I/O.
5. An I/O request completes.

Comparison among Scheduler

S. No.	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming.
4	It is almost absent or minimal in time sharing system	It is also minimal in time sharing system	It is a part of Time sharing systems.
5	It selects processes from pool and loads them into memory for execution	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.

Context Switch

A context switching is the mechanism to store and restore the state or context of a CPU in Process Control block so that a process execution can be resumed from the same point at a later time.



Context Switch

1. Switching the CPU to another process requires saving the state of the old process and loading the saved state for the new process. This task is known as a Context Switch.

2. The context of a process is represented in the Process Control Block(PCB) of a process; it includes the value of the CPU registers, the process state and memory-management information. When a context switch occurs, the Kernel saves the context of the old process in its PCB and loads the saved context of the new process scheduled to run.

Context Switch

3. Context switch time is pure overhead, because the system does no useful work while switching. Its speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instructions(such as a single instruction to load or store all registers). Typical speeds range from 1 to 1000 microseconds.

4. Context Switching has become such a performance bottleneck that programmers are using new structures(threads) to avoid it whenever and wherever possible.

Operation on Processes

1. Creation : Once the process is created, it will be ready and come into the ready queue (main memory) and will be ready for the execution.

2. Scheduling : Out of the many processes present in the ready queue, the Operating system chooses one process and start executing it. Selecting the process which is to be executed next, is known as scheduling.

3. Execution : Once the process is scheduled for the execution, the processor starts executing it. Process may come to the blocked or wait state during the execution then in that case the processor starts executing the other processes.

4. Deletion/killing : Once the purpose of the process gets over then the OS will kill the process. The Context of the process (PCB) will be deleted and the process gets terminated by the Operating system.

Process Creation

- **A process during its execution can create many new processes via system call(depending upon the OS).**
- **The creating process is called the parent while the created process is called the child.**
- **Each child process may in turn create new child process.**
- **Every process in the system is identified with a process identifier(PID) which is unique for each process.**

Process Creation

Why to create a child process?

How to create a process?

How the child process gets its resources?

Execution Possibilities for parent process

Address Space of child process

Process Creation

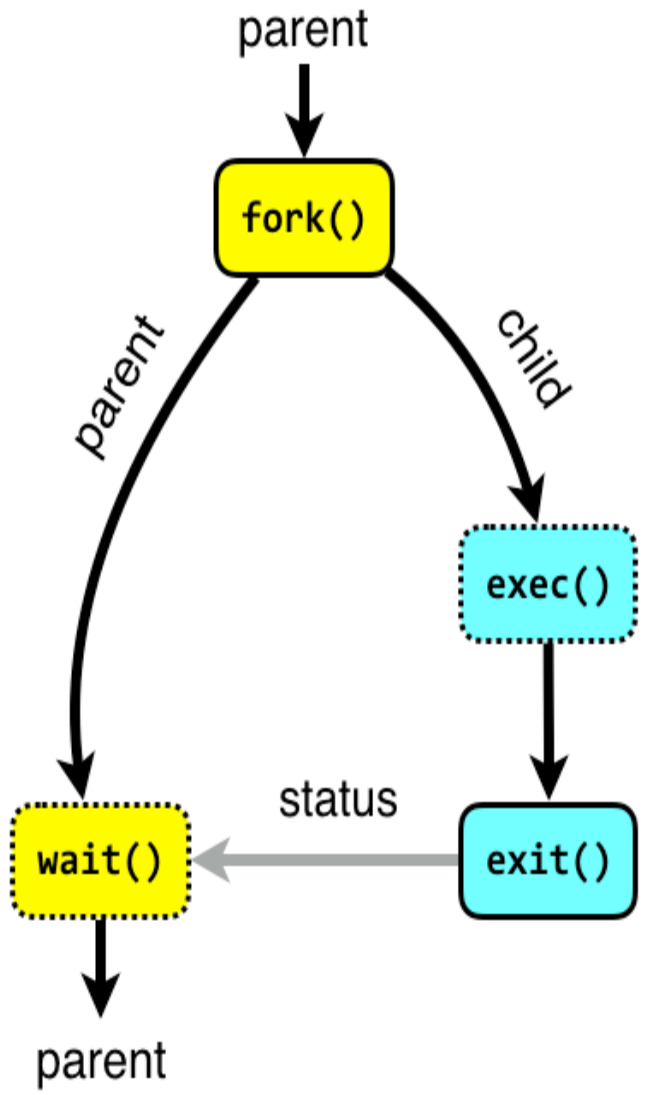
The following system calls are used for basic process management.

fork : A parent process uses fork to create a new child process. The child process is a copy of the parent. After fork, both parent and child executes the same program but in separate processes.

exec : Replaces the program executed by a process. The child may use exec after a fork to replace the process' memory space with a new program executable making the child execute a different program than the parent.

exit : Terminates the process with an exit status.

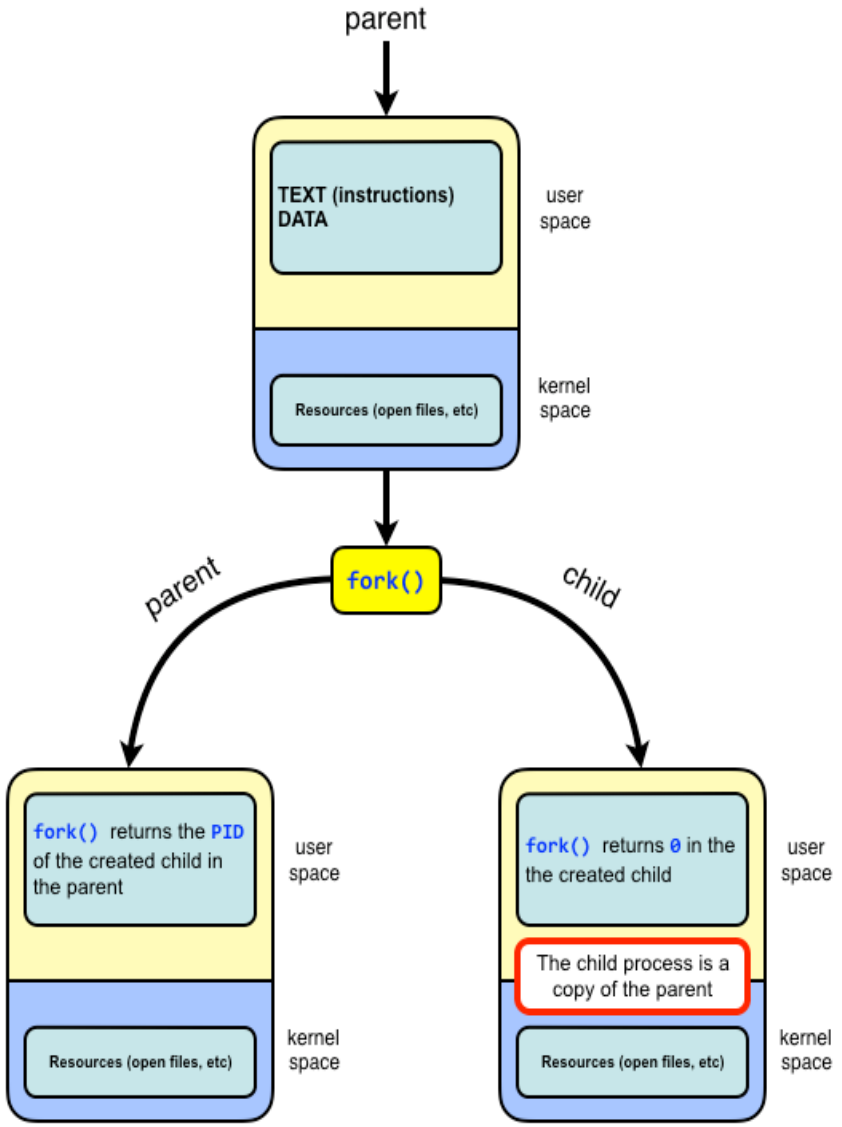
wait : The parent may use wait to suspend execution until a child terminates. Using wait the parent can obtain the exit status of a terminated child.



Process Creation

On success fork returns twice: once in the parent and once in the child. After calling fork, the program can use the fork return value to tell whether executing in the parent or child.

- If the return value is 0 the program executes in the new child process.
- If the return value is greater than zero, the program executes in the parent process and the return value is the process ID (PID) of the created child process.
- On failure fork returns -1.



```c
#include<stdio.h>
void main(int argc, char *argv[]) {
        int pid;
        /* Fork another process */
        pid = fork();
        if(pid < 0) {  //Error occurred
                fprintf(stderr, "Fork Failed");
                 exit(-1); }
        else if (pid == 0) {  //Child process
                execlp("/bin/ls","ls",NULL); }
         else { //Parent process
                //Parent will wait for the child to complete
                wait(NULL);
                printf("Child complete");
                exit(0);
                } }
```

Wait

The wait system call blocks the caller until one of its child process terminates. If the caller doesn't have any child processes, wait returns immediately without blocking the caller. Using wait the parent can obtain the exit status of the terminated child.

```
#include <sys/types.h>
#include <sys/wait.h>
pid_t wait(int *status);
```

status
If status **is not NULL,** wait **store the exit status of the terminated child in the int to which** status **points.**

Return value
On success, wait **returns the PID of the terminated child. On failure (no child),** wait **returns -1.**

CPU Scheduling

- **The goal of the multi-programming system is to keep the CPU busy at all times.**
- **In a uni-processor system whenever the CPU becomes idle it is allocated to a new process.**
- **Which process will be allocated the CPU is decided by the *short-term scheduler*.**
- **CPU scheduling means to select the next process to be allocated to the CPU whenever the CPU becomes idle.**
- **Other common names of CPU scheduling are *process scheduling* or *thread scheduling*.**

CPU Scheduling

The question is – ”When to call the short-term scheduler?”

There are four circumstances:

1. **A Process switches from running state to waiting state.**
2. **Process switches from running state to ready state.**
3. **Process switches from waiting state to ready state.**
4. **A Process terminates.**

Dispatcher

A Dispatcher is that module of the operating system which gives control of the CPU to the process. Short term scheduler only decides which process to allocate to CPU. Whereas the dispatcher actually allocates the CPU to that process.

It involves three steps:

- **First, it performs Context switching**
- **Second, Switching to user mode**
- **Finally, Jumping to the proper location in the user program to restart that process.**

Scheduling Algorithm's

Which is the best CPU scheduling algorithm?

This is a common question that comes to our mind. An Operating System uses different algorithms for CPU scheduling. Every algorithm can perform better depending upon the situation.

There are five points to decide which algorithm performs better than others.

1. **CPU utilization – It means how much the CPU is busy. That is, for a given duration of time how long was the CPU doing the processing. The CPU utilization should be high (40-90%)**
2. **Throughput – Throughput is the number of processes that complete execution in a given time.**
3. **Turnaround time – is the interval from the time of submission of the process to the time of its completion**

Scheduling Algorithm's

4. Waiting time – is the time spent by the process in the ready queue (waiting for the CPU)

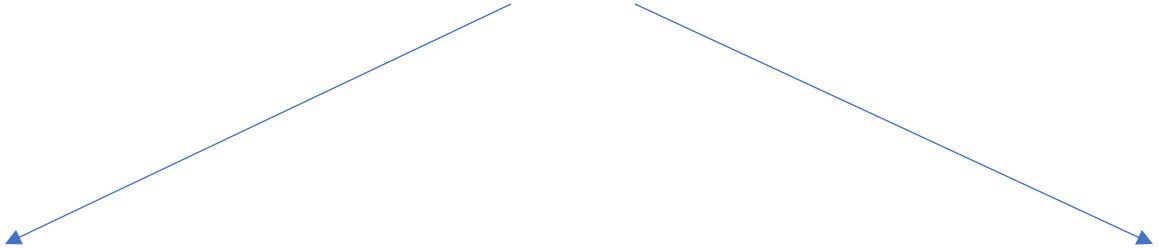
5. Response time – is the time from the submission of the process until it gives the first response.

Note: An algorithm is effective if it has high CPU utilization and throughput, whereas low turnaround, waiting and response time.

CPU Scheduling Algorithm's

- **Arrival Time:** ***Time at which process enters the ready queue or state.***
- **Burst Time:** ***Time required by a process to get executed on CPU.***
- **Completion Time:** ***Time at which process completes its execution.***
- **Turn around Time:** ***Completion Time – Arrival Time***
- **Waiting Time:** ***Turn-around Time – Burst Time***
- **Response Time: Time at which process get CPU for the** ***first time – Arrival Time***

CPU Scheduling Algorithm's



Pre-emptive Scheduling
Shortest Remaining Time First (SRTF)
Largest Remaining Time First (LRTF)
Round Robin
Priority Based

Non Pre-emptive Scheduling
First Come First Serve (FCFS)
Shortest Job First (SJF)
Largest Job First (LJF)
Multi-level Queue
Highest Response Ratio Next (HRRN)

First Come First Serve

- **It is a simple scheduling algorithm.**
- **The idea is that the process that comes first must use the resource first.**
- **It schedules according to the arrival time of the process.**
- **It states that process which request the CPU first is allocated the CPU first.**
- **It is implemented by using *FIFO (First Come First Serve) queue* and is a *Non Pre-emptive* scheduling algorithm.**

First Come First Serve

Example:

PROCESS	BURST TIME	WAITING TIME
P1	8	0
P2	3	8
P3	5	11
P4	3	16

Analysing the given processes, first the process P1 will be executed. Therefore, waiting time of process P1 will be 0.

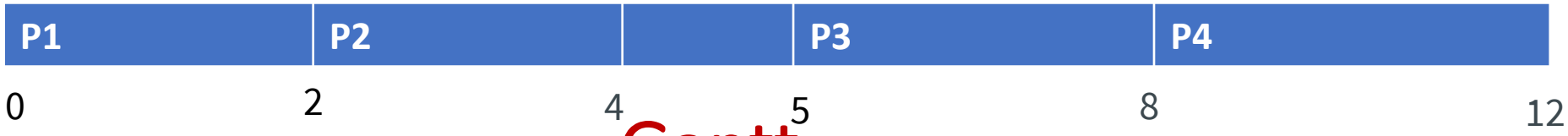
As, process P1 takes 8 units of time for completion, so waiting time for process P2 will be 8.

Likewise, waiting time of process P3 will be execution time of process P1 + execution time for process P2, i.e. (8 + 3) units = 11 units. Furthermore, for process P4 it will be the sum of execution times of P1, P2 and P3.

First Come First Serve

Example:

Process	Arrival Time	Burst Time	Completion Time	Turn-around Time	Waiting Time	Response Time
P1	0	2	2	2	0	0
P2	1	2	4	3	1	1
P3	5	3	8	3	0	0
P4	6	4	12	6	2	2



Gantt Chart

First Come First Serve

Advantages of FCFS :

- **Simple**
- **Easy, useful and understandable**
- **First come, first served.**

Convoy Effect

In convoy effect,

- ***Consider processes with higher burst time arrived before the processes with smaller burst time.***
- ***Then, smaller processes have to wait for a long time for longer processes to release the CPU.***

Disadvantages of FCFS :

- **Because of non pre-emptive scheduling, the process will continue to run until it is finished.**
- **As the scheduling is non pre-emptive so short processes which are at end of ready queue have to wait for a larger time making them starve leading to problem of starvation.**
- **Throughput is not efficient.**

First Come First Serve

Example :

Consider the following set of four processes. Their arrival time and time required to complete the execution are given in following table. Consider all time values in milliseconds.

Process	Arrival Time (T0)	Time required for completion (∆T)(CPU Burst Time)
P0	0	10
P1	1	6
P2	3	2
P3	5	4

First Come First Serve

Statistics:

Process	Arrival Time(T0)	CPU Burst Time(∆T)	Finish Time(T1)	Turnaround Time(TAT=T1-T0)	Waiting Time (WT=TAT-∆T)
P0	0	10	10	10	0
P1	1	6	16	15	9
P2	3	2	18	15	13
P3	5	4	22	17	13

Average Turnaround Time: (10+15+15+17)/4 = 57/4 = 14.25 ms.

Average Waiting Time: (0+9+13+13)/4 = 35/4 = 8.75 ms.

Shortest Job First (SJF)

- **Process with the shortest burst time is scheduled first.**
- **If two processes have same burst time, then FCFS is used to break the tie.**
- **SJF Scheduling can be used in both pre-emptive and non-preemptive mode.**
- **Pre-emptive mode of Shortest Job First is called as Shortest Remaining Time First (SRTF).**

Shortest Job First (SJF)

Example:

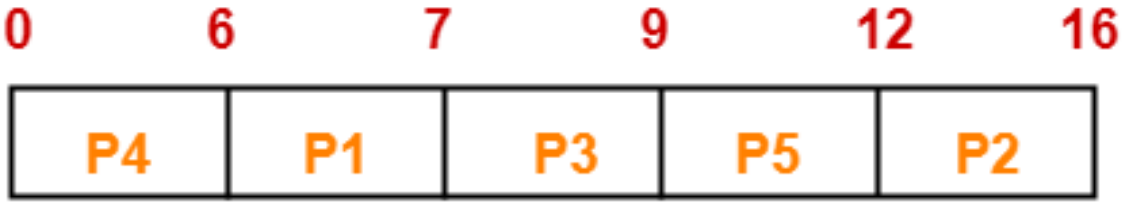
Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

If the CPU scheduling policy is SJF non-preemptive, calculate the average waiting time and average turn around time.

Shortest Job First (SJF)

Example:



Gantt Chart

- **Turn Around time = Exit time – Arrival time**
- **Waiting time = Turn Around time – Burst time**

Process Id	Exit time	Turn Around time	Waiting time
P1	7	7 – 3 = 4	4 – 1 = 3
P2	16	16 – 1 = 15	15 – 4 = 11
P3	9	9 – 4 = 5	5 – 2 = 3
P4	6	6 – 0 = 6	6 – 6 = 0
P5	12	12 – 2 = 10	10 – 3 = 7

- **Average Turn Around time = (4 + 15 + 5 + 6 + 10) / 5 = 40 / 5 = 8 unit**
- **Average waiting time = (3 + 11 + 3 + 0 + 7) / 5 = 24 / 5 = 4.8 unit**

Shortest Job First (SJF)

Example:

Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

If the CPU scheduling policy is SJF preemptive, calculate the average waiting time and average turn around time.

Shortest Job First (SJF)

Example:

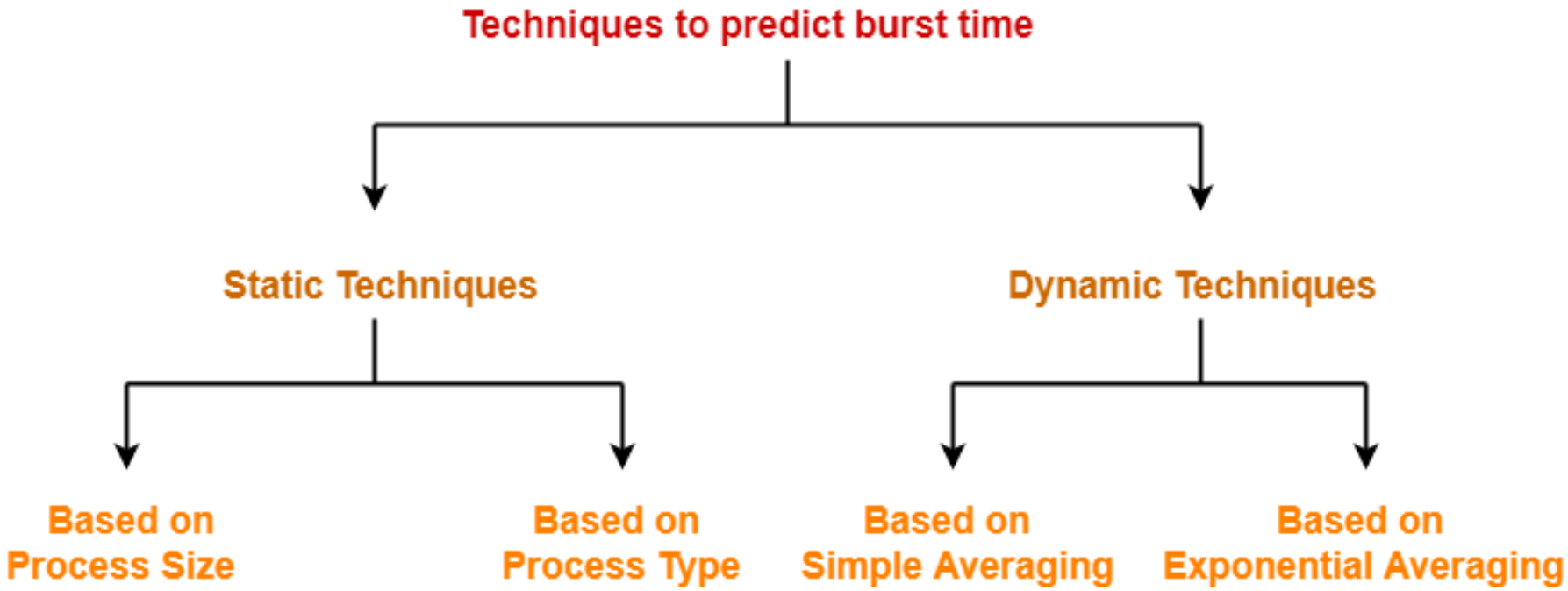
Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	Arrival time	Burst time
P1	0	7
P2	1	5
P3	2	3
P4	3	1
P5	4	2
P6	5	1

If the CPU scheduling policy is shortest remaining time first, calculate the average waiting time and average turn around time.

Technique to predict the Burst Time

There are several techniques which try to predict the burst time for the processes so that the algorithm can be implemented.



Static Techniques to predict the Burst Time

1. Based on Process Size

- **This technique predicts the burst time for a process based on its size.**
- **Burst time of the already executed process of similar size is taken as the burst time for the process to be executed.**

Example-

- **Consider a process of size 200 KB took 20 units of time to complete its execution.**
- **Then, burst time for any future process having size around 200 KB can be taken as 20 units.**

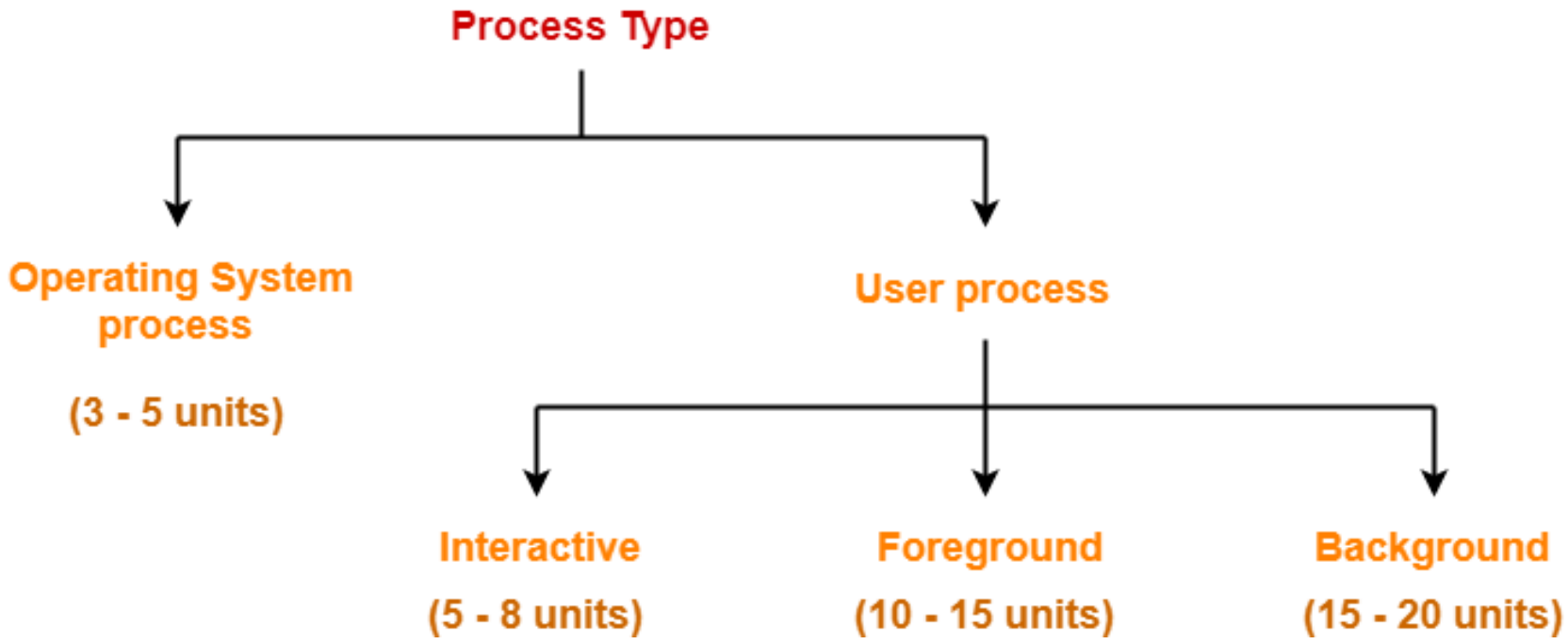
NOTE

- *The predicted burst time may not always be right.*
- *This is because the burst time of a process also depends on what kind of a process it is.*

Static Techniques to predict the Burst Time

2. <u>Based on Process Type</u>

- **This technique predicts the burst time for a process based on its type.**
- **The following figure shows the burst time assumed for several kinds of processes.**



Dynamic Techniques to predict the Burst Time

1. **Based on Simple Averaging**

- Burst time for the process to be executed is taken as the average of all the processes that are executed till now.
- Given n processes P_1, P_2, … , P_n and burst time of each process P_i as t_i, then predicted burst time for process P_{n+1} is given as-

$$T_{n+1} = \frac{1}{n} \sum_{i=1}^{n} t_i$$

Dynamic Techniques to predict the Burst Time

Given n processes P_1, P_2, … , P_n and burst time of each process P_i as t_i. Then, predicted burst time for process P_{n+1} is given as-

$$T_{n+1} = \alpha t_n + (1 - \alpha) T_n$$

where-

- **α is called smoothening factor (0<= α <=1)**
- **t_n = actual burst time of process P_n**
- **T_n = Predicted burst time for process P_n**

Example

Calculate the predicted burst time using exponential averaging for the fifth process if the predicted burst time for the first process is 10 units and actual burst time of the first four processes is 4, 8, 6 and 7 units respectively. Given α = 0.5.

Solution-

Given:

- **Predicted burst time for 1st process = 10 units**
- **Actual burst time of the first four processes = 4, 8, 6, 7**
- **α = 0.5**

Example

Predicted Burst Time for 2nd Process-

Predicted burst time for 2nd process
= α x Actual burst time of 1st process + (1-α) x Predicted burst time for 1st process
= 0.5 x 4 + 0.5 x 10
= 2 + 5
= 7 units

Predicted Burst Time for 3rd Process-

Predicted burst time for 3rd process
= α x Actual burst time of 2nd process + (1-α) x Predicted burst time for 2nd process
= 0.5 x 8 + 0.5 x 7
= 4 + 3.5
= 7.5 units

Example

Predicted Burst Time for 4th Process-

Predicted burst time for 4th process
= α x Actual burst time of 3rd process + (1-α) x Predicted burst time for 3rd process
= 0.5 x 6 + 0.5 x 7.5
= 3 + 3.75
= 6.75 units

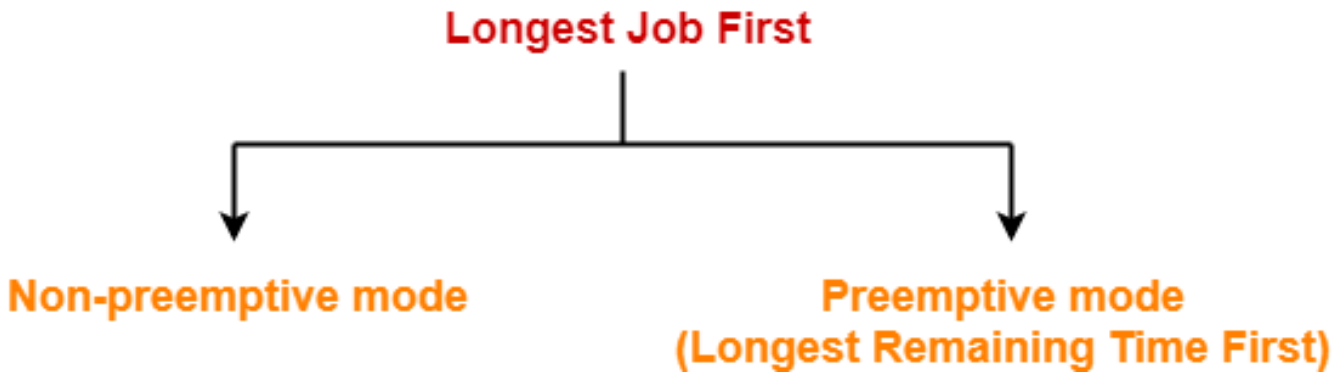
Predicted Burst Time for 5th Process-

Predicted burst time for 5th process
= α x Actual burst time of 4th process + (1-α) x Predicted burst time for 4th process
= 0.5 x 7 + 0.5 x 6.75
= 3.5 + 3.375
= 6.875 units

Longest Job First Algorithm

In LJF Scheduling,

- **Out of all the available processes, CPU is assigned to the process having largest burst time.**
- **In case of a tie, it is broken by FCFS Scheduling.**



- **LJF Scheduling can be used in both preemptive and non-preemptive mode.**
- **Preemptive mode of Longest Job First is called as Longest Remaining Time First (LRTF).**

Longest Job First Algorithm

Advantages-

- **No process can complete until the longest job also reaches its completion.**
- **All the processes approximately finishes at the same time.**

Disadvantages-

- **The waiting time is high.**
- **Processes with smaller burst time may starve for CPU.**

Longest Job First Algorithm

Example:

Consider the set of 5 processes whose arrival time and burst time are given below. If the CPU scheduling policy is LJF non-preemptive, calculate the average waiting time and average turn around time.

Process Id	Arrival time	Burst time
P1	0	3
P2	1	2
P3	2	4
P4	3	5
P5	4	6

Longest Job First Algorithm

Example:

Consider the set of 5 processes whose arrival time and burst time are given below. If the CPU scheduling policy is LJF non-preemptive, calculate the average waiting time and average turn around time.

Process Id	Arrival time	Burst time
P1	0	3
P2	1	2
P3	2	4
P4	3	5
P5	4	6

Longest Job First Algorithm

Process Id	Arrival time	Burst time
P1	0	3
P2	1	2
P3	2	4
P4	3	5
P5	4	6

Solution:



Gantt Chart

- Turn Around time = Exit time – Arrival time
- Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting time
P1	3	3 – 0 = 3	3 – 3 = 0
P2	20	20 – 1 = 19	19 – 2 = 17
P3	18	18 – 2 = 16	16 – 4 = 12
P4	8	8 – 3 = 5	5 – 5 = 0
P5	14	14 – 4 = 10	10 – 6 = 4

- Average Turn Around time = (3 + 19 + 16 + 5 + 10) / 5 = 53 / 5 = 10.6 unit
- Average waiting time = (0 + 17 + 12 + 0 + 4) / 5 = 33 / 5 = 6.6 unit

Longest Job First Algorithm

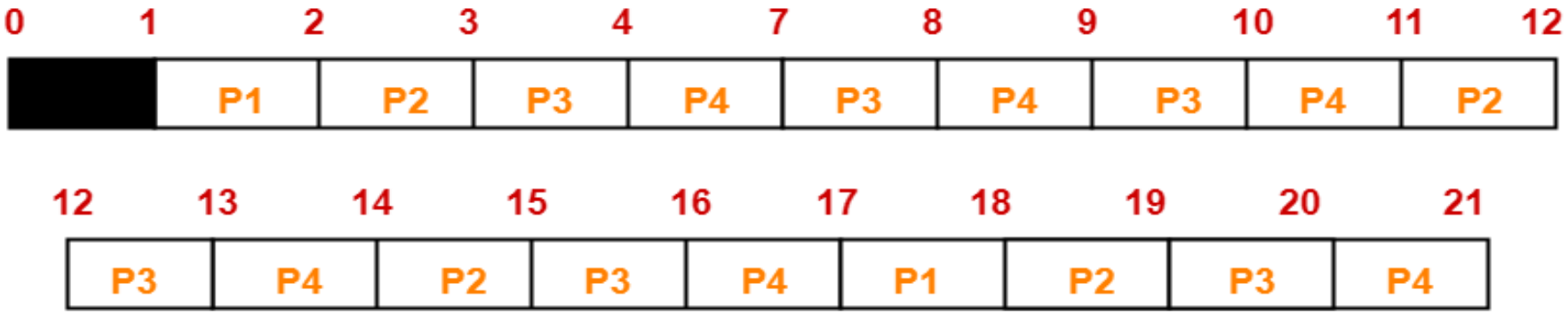
Example:

Consider the set of 4 processes whose arrival time and burst time are given below. If the CPU scheduling policy is LJF preemptive, calculate the average waiting time and average turn around time.

Process Id	Arrival time	Burst time
P1	1	2
P2	2	4
P3	3	6
P4	4	8

Longest Job First Algorithm

Solution:



Gantt Chart

Process Id	Exit time	Turn Around time	Waiting time
P1	18	18 – 1 = 17	17 – 2 = 15
P2	19	19 – 2 = 17	17 – 4 = 13
P3	20	20 – 3 = 17	17 – 6 = 11
P4	21	21 – 4 = 17	17 – 8 = 9

- Average Turn Around time = (17 + 17 + 17 + 17) / 4 = 68 / 4 = 17 unit
- Average waiting time = (15 + 13 + 11 + 9) / 4 = 48 / 4 = 12 unit

Longest Job First Algorithm

Example:

Consider the set of 4 processes whose arrival time and burst time are given below. If the CPU scheduling policy is LJF pre-emptive, calculate the average waiting time and average turn around time.

Process Id	Arrival time	Burst time
P1	1	2
P2	2	4
P3	3	6
P4	4	8

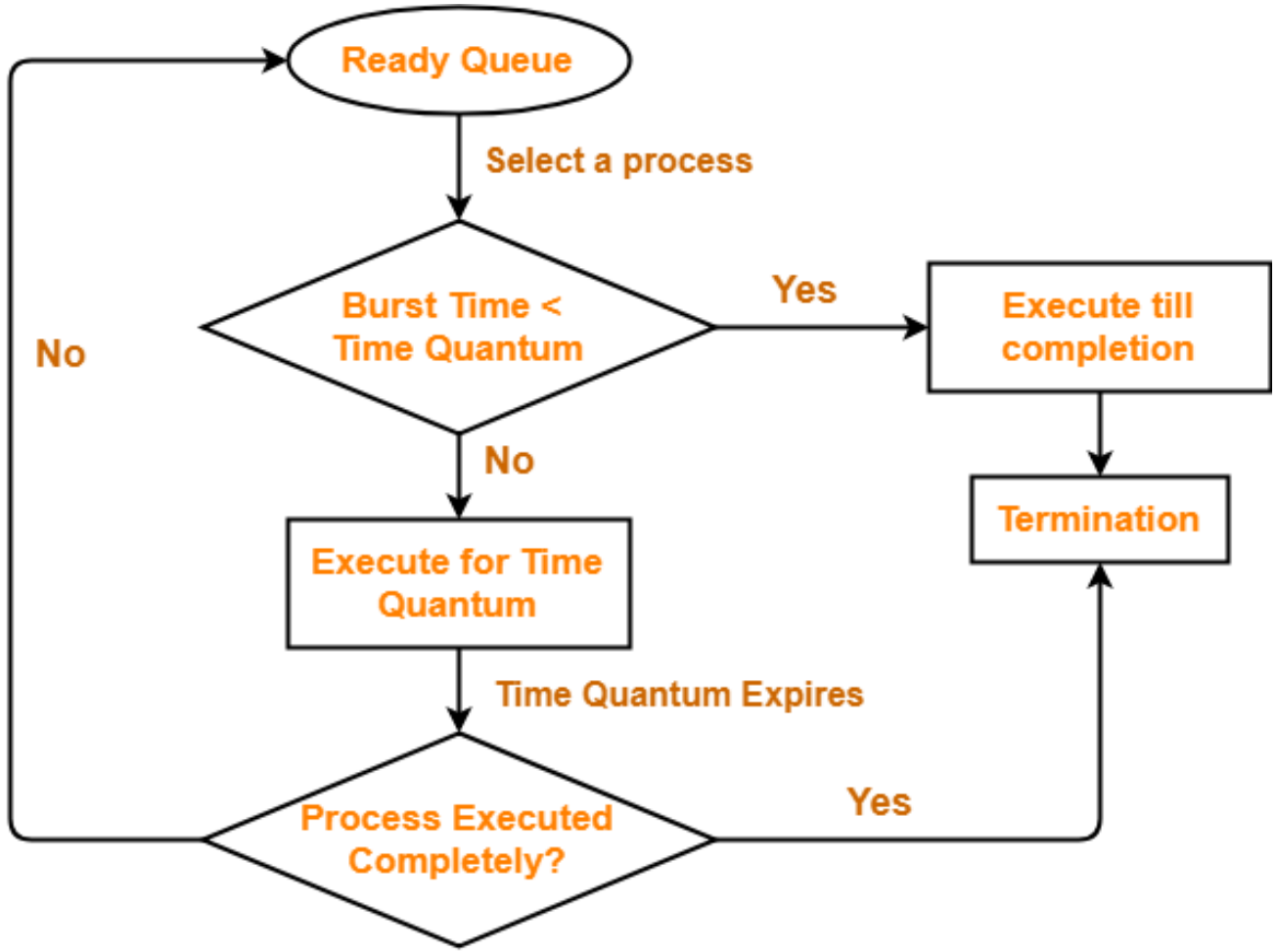
Round Robin Scheduling Algorithm

In Round Robin Scheduling,

- **CPU is assigned to the process based on FCFS for a fixed amount of time.**
- **This fixed amount of time is called as time quantum or time slice.**
- **After the time quantum expires, the running process is preempted and sent to the ready queue.**
- **Then, the processor is assigned to the next arrived process.**
- **It is always preemptive in nature.**

Round Robin Scheduling is FCFS Scheduling with pre-emptive mode.

Round Robin Scheduling Algorithm



Round Robin Scheduling

Round Robin Scheduling Algorithm

Advantages-

- **It gives the best performance in terms of average response time.**
- **It is best suited for time sharing system, client server architecture and interactive system.**

Disadvantages-

- **It leads to starvation for processes with larger burst time as they have to repeat the cycle many times.**
- **Its performance heavily depends on time quantum.**
- **Priorities can not be set for the processes.**

Round Robin Scheduling Algorithm

Note-01:

With decreasing value of time quantum,

- ***Number of context switch increases***
- ***Response time decreases***
- ***Chances of starvation decreases***

Thus, smaller value of time quantum is better in terms of response time.

Round Robin Scheduling Algorithm

Note-02:

With increasing value of time quantum,

- ***Number of context switch decreases***
- ***Response time increases***
- ***Chances of starvation increases***

Thus, higher value of time quantum is better in terms of number of context switch.

Round Robin Scheduling Algorithm

Note-03:

- **With increasing value of time quantum, Round Robin Scheduling tends to become FCFS Scheduling.**
- **When time quantum tends to infinity, Round Robin Scheduling becomes FCFS Scheduling.**

Note-04:

- **The performance of Round Robin scheduling heavily depends on the value of time quantum.**
- **The value of time quantum should be such that it is neither too big nor too small.**

Round Robin Scheduling Algorithm

Example: Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	Arrival time	Burst time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

If the CPU scheduling policy is Round Robin with time quantum = 2 unit, calculate the average waiting time and average turn around time.

Round Robin Scheduling Algorithm

Example: Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	Arrival time	Burst time
P1	0	4
P2	1	5
P3	2	2
P4	3	1
P5	4	6
P6	6	3

If the CPU scheduling policy is Round Robin with time quantum = 2 unit, calculate the average waiting time and average turn around time.

Round Robin Scheduling Algorithm

Example: Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	Arrival time	Burst time
P1	5	5
P2	4	6
P3	3	7
P4	1	9
P5	2	2
P6	6	3

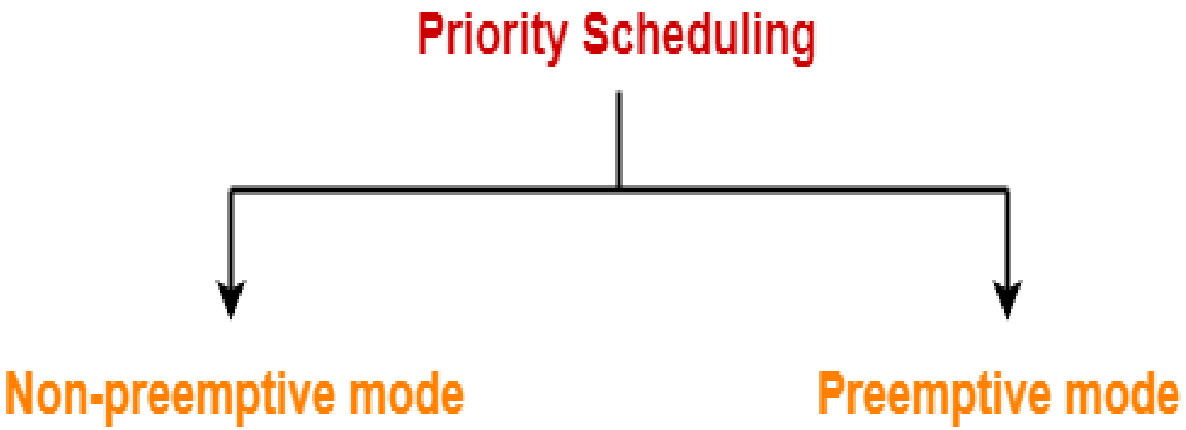
If the CPU scheduling policy is Round Robin with time quantum = 3 unit, calculate the average waiting time and average turn around time.

Priority Scheduling

In Priority Scheduling,

- **Out of all the available processes, CPU is assigned to the process having the highest priority.**
- **In case of a tie, it is broken by FCFS Scheduling.**

- **Priority Scheduling can be used in both preemptive and non-preemptive mode.**



Priority Scheduling

Advantages-

- **It considers the priority of the processes and allows the important processes to run first.**
- **Priority scheduling in pre-emptive mode is best suited for real time operating system.**

Disadvantages-

- **Processes with lesser priority may starve for CPU.**
- **There is no idea of response time and waiting time.**

Priority Scheduling

Note-01:

- **The waiting time for the process having the highest priority will always be zero in preemptive mode.**
- **The waiting time for the process having the highest priority may not be zero in non-preemptive mode.**

Note-02:

Priority scheduling in preemptive and non-preemptive mode behaves exactly same under following conditions-

- **The arrival time of all the processes is same**
- **All the processes become available**

Priority Scheduling

Example: Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	Arrival time	Burst time	Priority
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5

If the CPU scheduling policy is priority non-preemptive, calculate the average waiting time and average turn around time. (Higher number represents higher priority)

Priority Scheduling

Solution:



Gantt Chart

Process Id	Exit time	Turn Around time	Waiting time
P1	4	4 – 0 = 4	4 – 4 = 0
P2	15	15 – 1 = 14	14 – 3 = 11
P3	12	12 – 2 = 10	10 – 1 = 9
P4	9	9 – 3 = 6	6 – 5 = 1
P5	11	11 – 4 = 7	7 – 2 = 5

- **Average Turn Around time = (4 + 14 + 10 + 6 + 7) / 5 = 41 / 5 = 8.2 unit**
- **Average waiting time = (0 + 11 + 9 + 1 + 5) / 5 = 26 / 5 = 5.2 unit**

Priority Scheduling

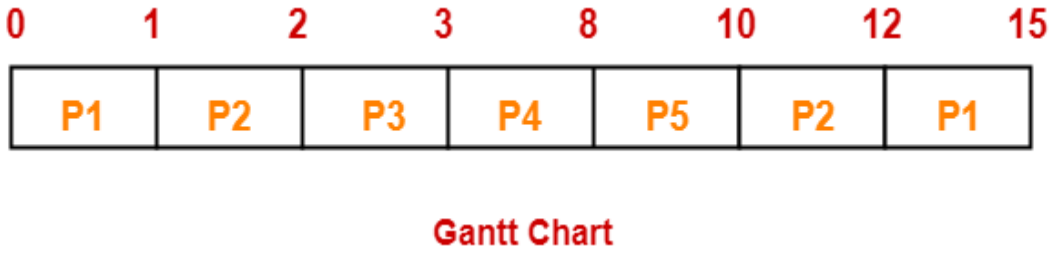
Example: Consider the set of 5 processes whose arrival time and burst time are given below

Process Id	Arrival time	Burst time	Priority
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5

If the CPU scheduling policy is priority pre-emptive, calculate the average waiting time and average turn around time. (Higher number represents higher priority)

Priority Scheduling

Solution:



Gantt Chart

Process Id	Exit time	Turn Around time	Waiting time
P1	15	15 – 0 = 15	15 – 4 = 11
P2	12	12 – 1 = 11	11 – 3 = 8
P3	3	3 – 2 = 1	1 – 1 = 0
P4	8	8 – 3 = 5	5 – 5 = 0
P5	10	10 – 4 = 6	6 – 2 = 4

- **Average Turn Around time = (15 + 11 + 1 + 5 + 6) / 5 = 38 / 5 = 7.6 unit**
- **Average waiting time = (11 + 8 + 0 + 0 + 4) / 5 = 23 / 5 = 4.6 unit**

Multilevel Queue Scheduling

Each algorithm supports a different process, but in a general system, some processes require scheduling using a priority algorithm. While some processes want to stay in the system ***(interactive processes),*** others are ***background processes*** whose execution can be delayed.

Each queue has its own scheduling algorithm

- **foreground – RR**
- **background – FCFS**

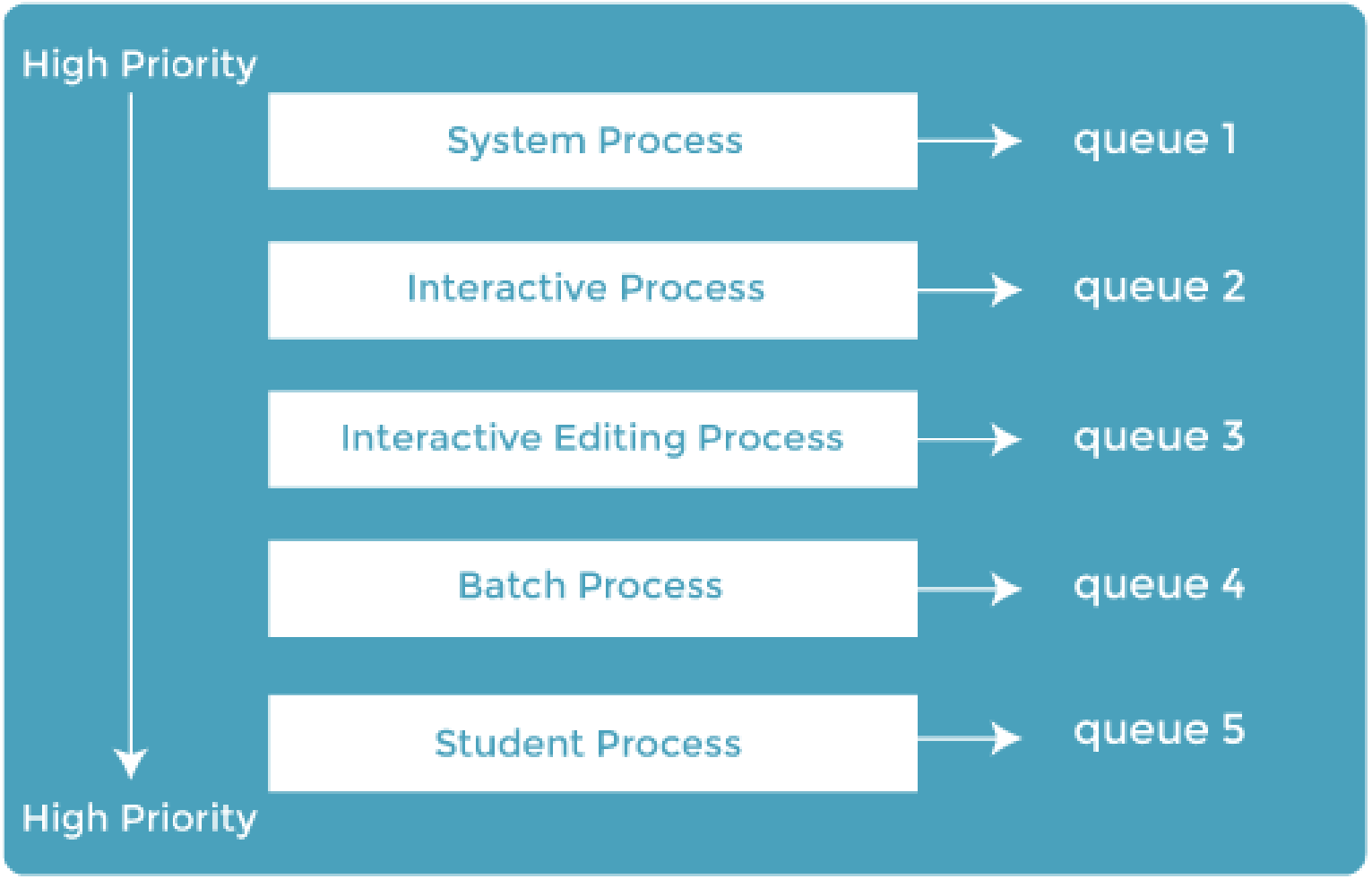
Scheduling must be done between the queues

- **Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.**
- **Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR**
- **20% to background in FCFS**

Multilevel Queue Scheduling

- **The ready queue has been partitioned into seven different queues using the multilevel queue scheduling technique.**
- **These processes are assigned to one queue based on their priority, such as memory size, process priority, or type. The method for scheduling each queue is different.**
- **Some queues are utilized for the foreground process, while others are used for the background process.**
- **The *foreground queue may* be scheduled using a *round-robin method,* and the *background queue can* be scheduled using an *FCFS* strategy.**

Multilevel Queue Scheduling



Multilevel Queue Scheduling

Advantages

1. You can use multilevel queue scheduling to apply different scheduling methods to distinct processes.

2. It will have low overhead in terms of scheduling.

Disadvantages

1. There is a risk of starvation for lower priority processes.

2. It is rigid in nature.

Multilevel Queue Scheduling

Example: Consider the four processes listed in the table below under multilevel queue scheduling. The queue number denotes the process's queue.

Process	Arrival Time	CPU Burst Time	Queue Number
P1	0	4	1
P2	0	3	1
P3	0	8	2
P4	10	5	1

Queue 1 has a higher priority than queue 2. Round Robin is used in queue 1 (Time Quantum = 2), while FCFS is used in queue 2.

Multilevel Queue Scheduling

Solution: Both queues have been processed at the start. Therefore, queue 1 (P1, P2) runs first (due to greater priority) in a round-robin way and finishes after 7 units.

1.The process in queue 2 (Process P3) starts running (since there is no process in queue 1), but while it is executing, P4 enters queue 1 and interrupts P3, and then P3 takes the CPU and finishes its execution.



Practice Set

The given processes are being scheduled using a pre-emptive, round-robin scheduling algorithm. Each process is assigned a numerical priority, with a higher number indicating a higher relative priority.

In addition to the processes listed, the system also has an ***idle task*** (which consumes no CPU resources and is identified as *Pidle*). This task has priority 0 and is scheduled whenever the system has no other available processes to run. The length of a time quantum is 10 units.

If a process is pre-empted by a higher-priority process, the pre-empted process is placed at the end of the queue.

Thread	Priority	Burst	Arrival
P_1	40	20	0
P_2	30	25	25
P_3	30	25	30
P_4	35	15	60
P_5	5	10	100
P_6	10	10	105

a. Show the scheduling order of the processes using a Gantt chart.
b. What is the turnaround time for each process?
c. What is the waiting time for each process?
d. What is the CPU utilization rate?

Practice Set

Consider four processes P, Q, R, and S scheduled on a CPU as per round robin algorithm with a time quantum of 4 units. The processes arrive in the order P, Q, R, S, all at time t = 0. There is exactly one context switch from S to Q, exactly one context switch from R to Q, and exactly two context switches from Q to R. There is no context switch from S to P. Switching to a ready process after the termination of another process is also considered a context switch. Which one of the following is NOT possible as CPU burst time (in time units) of these processes?

A. P = 4, Q = 10, R = 6, S = 2

B. P = 2, Q = 9, R = 5, S = 1

C. P = 4, Q = 12, R = 5, S = 4

D. P = 3, Q = 7, R = 7, S = 3

GATE CSE 2022 Operating System

Practice Problem

Three processes arrive at time zero with CPU bursts of 16, 20 and 10 milliseconds. If the scheduler has prior knowledge about the length of the CPU bursts, the minimum achievable average waiting time for these three processes in a non-preemptive scheduler (rounded to nearest integer) is _____ milliseconds.

A 18.66

B 12

C 15

D 9

GATE CSE 2021 SET-1 Operating System

Practice Problem

Three CPU-bound tasks, with execution times of 15, 12 and 5 time units respectively arrive at times 0, t and 8, respectively. If the operating system implements a shortest remaining time first scheduling algorithm, what should be the value of t to have 4 context switches? Ignore the context switches at time 0 and at the end.

A. $0 < t < 3$

B. $t=0$

C. $t <= 3$

D. $3 < t < 8$

ISRO CSE 2020 Operating System

Practice Problem

Consider the following set of processes, assumed to have arrived at time 0. Consider the CPU scheduling algorithms Shortest Job First (SJF) and Round Robin (RR). For RR, assume that the processes are scheduled in the order P_1, P_2, P_3, P_4.

Processes	P_1	P_2	P_3	P_4
Burst time (in ms)	8	7	2	4

If the time quantum for RR is 4 ms, then the absolute value of the difference between the average turnaround times (in ms) of SJF and RR (round off to 2 decimal places is_____

A 10.5

B 15.75

C 5.25

D 4.25

GATE CSE 2020 Operating System

Practice Problem

Consider the following four processes with arrival times (in milliseconds) and their length of CPU burst (in milliseconds) as shown below:

Process	P1	P2	P3	P4
Arrival time	0	1	3	4
CPU burst time	3	1	3	Z

These processes are run on a single processor using preemptive Shortest Remaining Time First scheduling algorithm. If the average waiting time of the processes is 1 millisecond, then the value of Z is _____.

A 1

B 2

C 3

D 4

GATE CSE 2019 Operating System

Practice Problem

Consider a uniprocessor system executing three tasks T1, T2 and T3, each of which is composed of an infinite sequence of jobs (or instances) which arrive periodically at intervals of 3, 7 and 20 milliseconds, respectively. The priority of each task is the inverse of its period, and the available tasks are scheduled in order of priority, with the highest priority task scheduled first. Each instance of T1, T2 and T3 requires an execution time of 1, 2 and 4 milliseconds, respectively. Given that all tasks initially arrive at the beginning of the 1st millisecond and task preemptions are allowed, the first instance of T3 completes its execution at the end of _____ milliseconds.

A 10

B 12

C 13

D 15

GATE CSE 2015 SET-1 Operating System

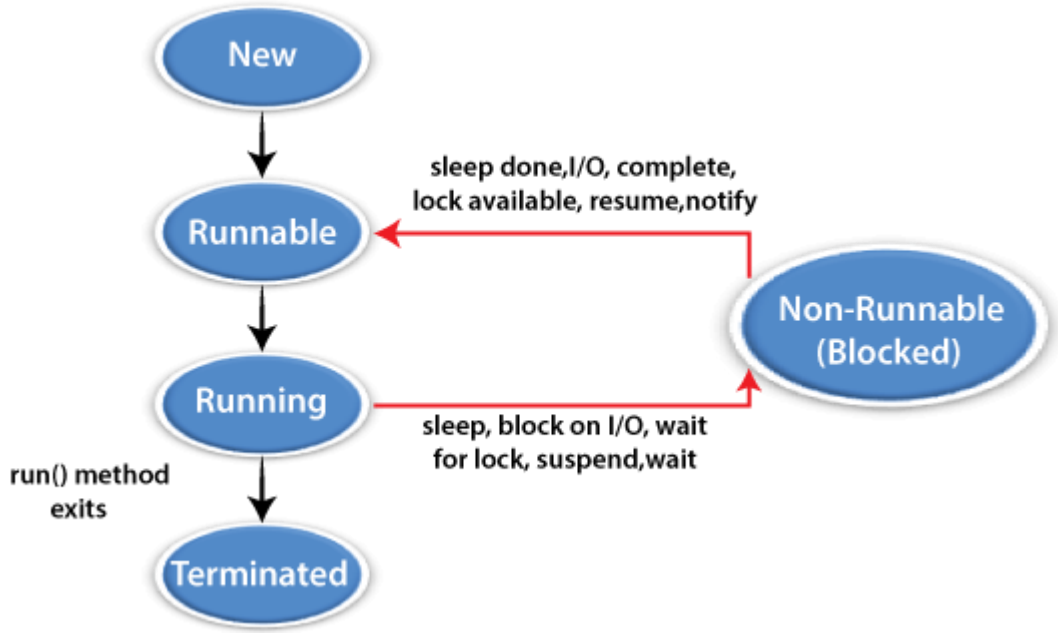
Thread

What are Threads?

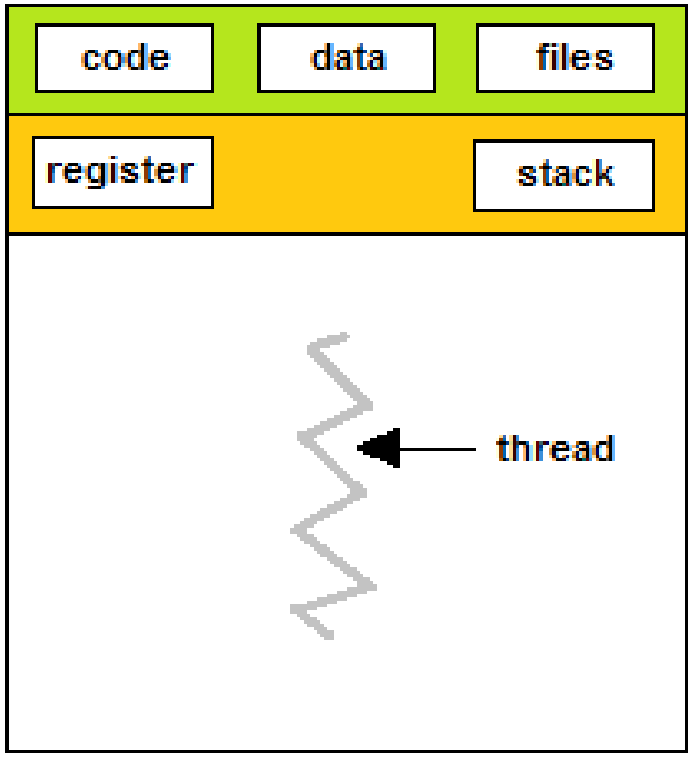
Thread is an execution unit that consists of its own program counter, a stack, and a set of registers where the program counter mainly keeps track of which instruction to execute next, a set of registers mainly hold its current working variables, and a stack mainly contains the history of execution.

- **Threads are also known as Lightweight processes.**
- **Threads are a popular way to improve the performance of an application through parallelism.**
- **As each thread has its own independent resource for process execution; thus, Multiple processes can be executed parallelly by increasing the number of threads.**
- **It is important to note here that each thread belongs to exactly one process and outside a process no threads exist.**
- **Threads provide a suitable foundation for the parallel execution of applications on shared-memory multiprocessors**

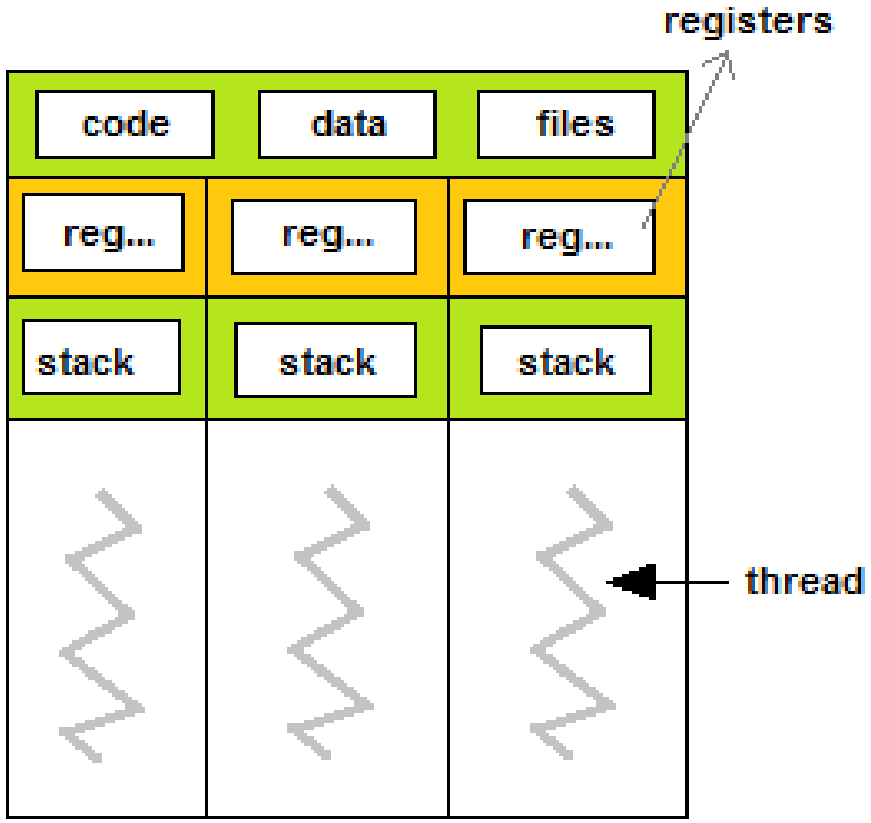
Thread Model



Thread



single-threaded process



multithreaded process

How Threads and Processes Work Step By Step

Here's what happens when you open an application on your computer.

- **The program starts out as a text file of programming code.**
- **The program is compiled or interpreted into binary form.**
- **The program is loaded into memory.**
- **The program becomes one or more running processes. Processes are typically independent of one another.**
- **Threads exist as the subset of a process.**
- **Threads can communicate with each other more easily than processes can.**
- **Threads are more vulnerable to problems caused by other threads in the same process.**

Process vs Thread

ASPECT	PROCESSES	THREADS
Definition	Independent programs with their own memory space.	Lightweight, smaller units of a process, share memory.
Creation Overhead	Higher overhead due to separate memory space.	Lower overhead as they share the same memory space.
Isolation	Processes are isolated from each other.	Threads share the same memory space.
Resource Allocation	Each process has its own set of system resources.	Threads share resources within the same process.
Independence	Processes are more independent of each other.	Threads are dependent on each other within a process.
Failure Impact	A failure in one process does not directly affect others.	A failure in one thread can affect others in the same process.
Sychronization	Less need from synchronization, as processes are isolated.	Requires careful synchronization due to shared resources.
Example Use Cases	Running multiple independent applications.	Multithreading within a single application for parallelism.
Memory Usage	Typically consumes more memory.	Consumes less memory compared to processes.

Advantages of Thread

1.Responsiveness

2.Resource sharing, hence allowing better utilization of resources.

3.Economy. Creating and managing threads becomes easier.

4.Scalability. One thread runs on one CPU. In Multithreaded processes, threads can be distributed over a series of processors to scale.

5.Context Switching is smooth. Context switching refers to the procedure followed by the CPU to change from one task to another.

6.Enhanced Throughput of the system.

Let us take an example for this: suppose a process is divided into multiple threads, and the function of each thread is considered as one job, then the number of jobs completed per unit of time increases which then leads to an increase in the throughput of the system.

Types of Thread

There are two types of threads:

User threads are above the kernel and without kernel support. These are the threads that application programmers use in their programs.

Kernel threads are supported within the kernel of the OS itself. All modern OSs support kernel-level threads, allowing the kernel to perform multiple simultaneous tasks and/or to service multiple kernel system calls simultaneously.

Types of Thread

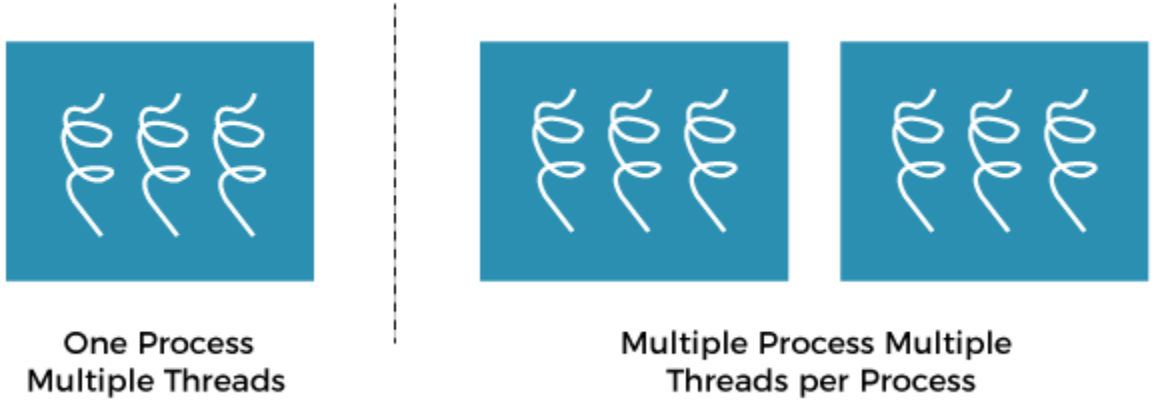
User Level threads	Kernel Level Threads
These threads are implemented by users.	These threads are implemented by Operating systems
These threads are not recognized by operating systems,	These threads are recognized by operating systems,
In User Level threads, the Context switch requires no hardware support.	In Kernel Level threads, hardware support is needed.
These threads are mainly designed as dependent threads.	These threads are mainly designed as independent threads.
In User Level threads, if one user-level thread performs a blocking operation then the entire process will be blocked.	On the other hand, if one kernel thread performs a blocking operation then another thread can continue the execution.
Example of User Level threads: Java thread, POSIX threads.	Example of Kernel level threads: Window Solaris.
Implementation of User Level thread is done by a thread library and is easy.	While the Implementation of the kernel-level thread is done by the operating system and is complex.
This thread is generic in nature and can run on any operating system.	This is specific to the operating system.

Multithreading

Multithreading is the ability of a program or an operating system to enable more than one user at a time without requiring multiple copies of the program running on the computer. Multithreading can also handle multiple requests from the same user.

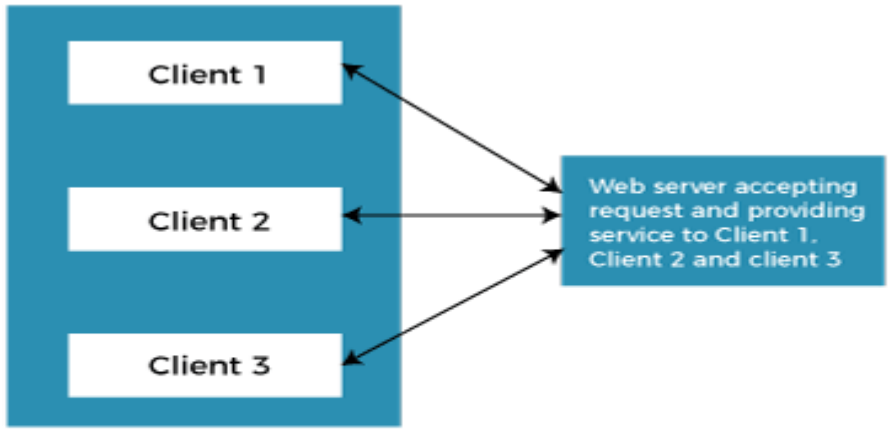
Multithreading allows the application to divide its task into individual threads. In multi-threads, the same process or task can be done by the number of threads, or we can say that there is more than one thread to perform the task in multithreading. With the use of multithreading, multitasking can be achieved.

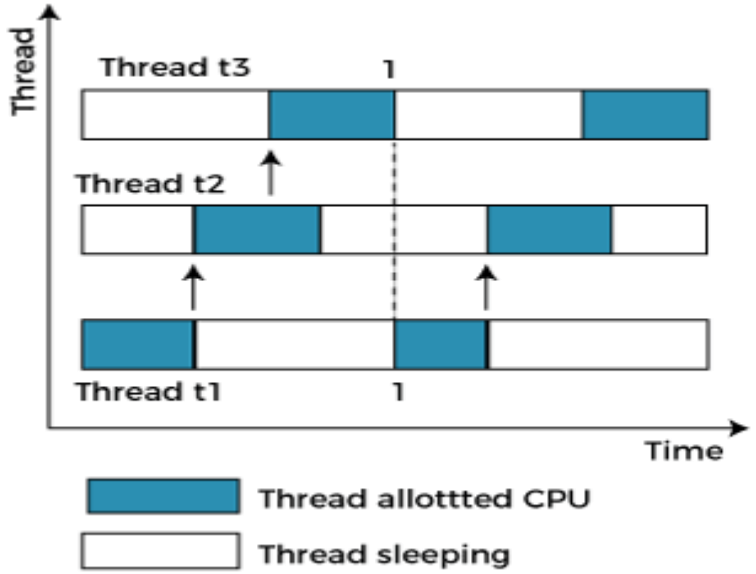
Multithreading



The main drawback of single threading systems is that only one task can be performed at a time, so to overcome the drawback of this single threading, there is multithreading that allows multiple tasks to be performed.

Multithreading : Example





In the above example, client1, client2, and client3 are accessing the web server without any waiting. In multithreading, several tasks can run at the same time.

Multithreading Models

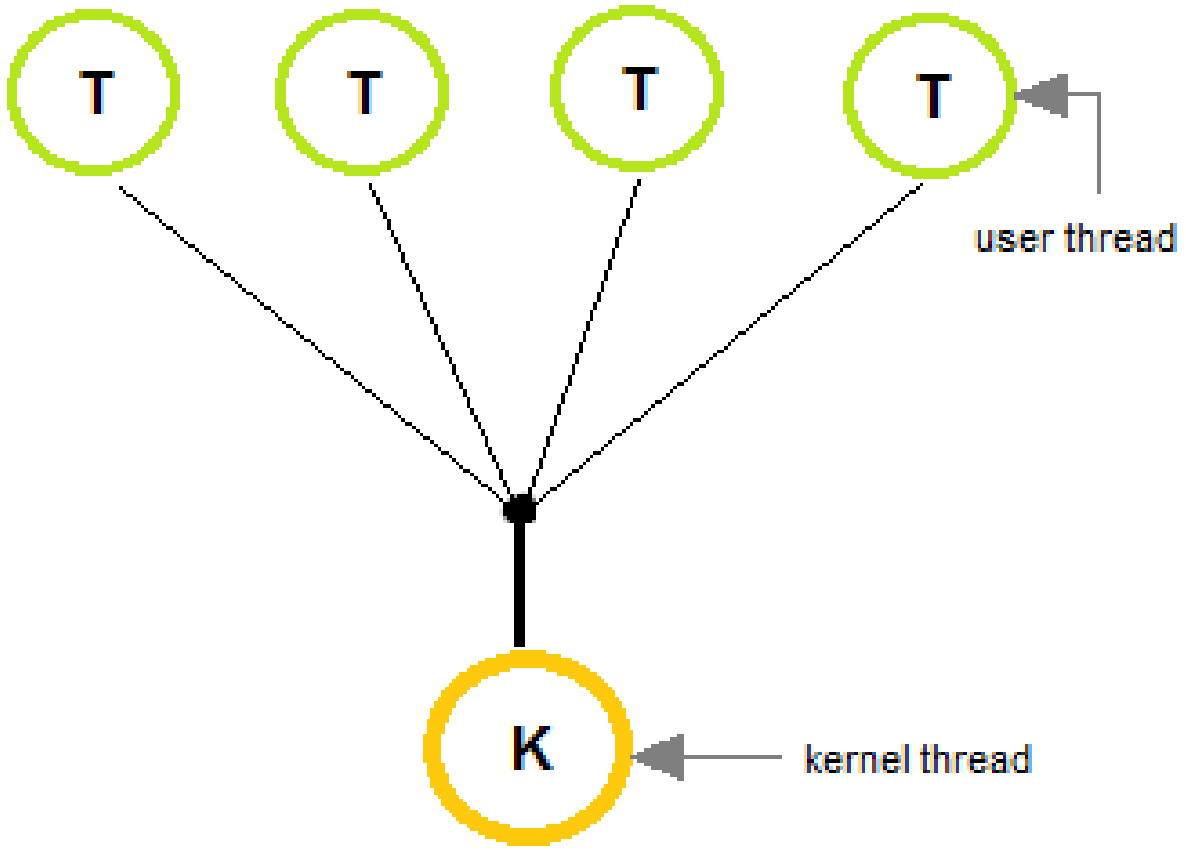
The user threads must be mapped to kernel threads, by one of the following strategies:

- **Many to One Model**
- **One to One Model**
- **Many to Many Model**

Many to One Model

- **In the many to one model, many user-level threads are all mapped onto a single kernel thread.**
- **Thread management is handled by the thread library in user space, which is efficient in nature.**
- **In this case, if user-level thread libraries are implemented in the operating system in some way that the system does not support them, then the Kernel threads use this many-to-one relationship model.**

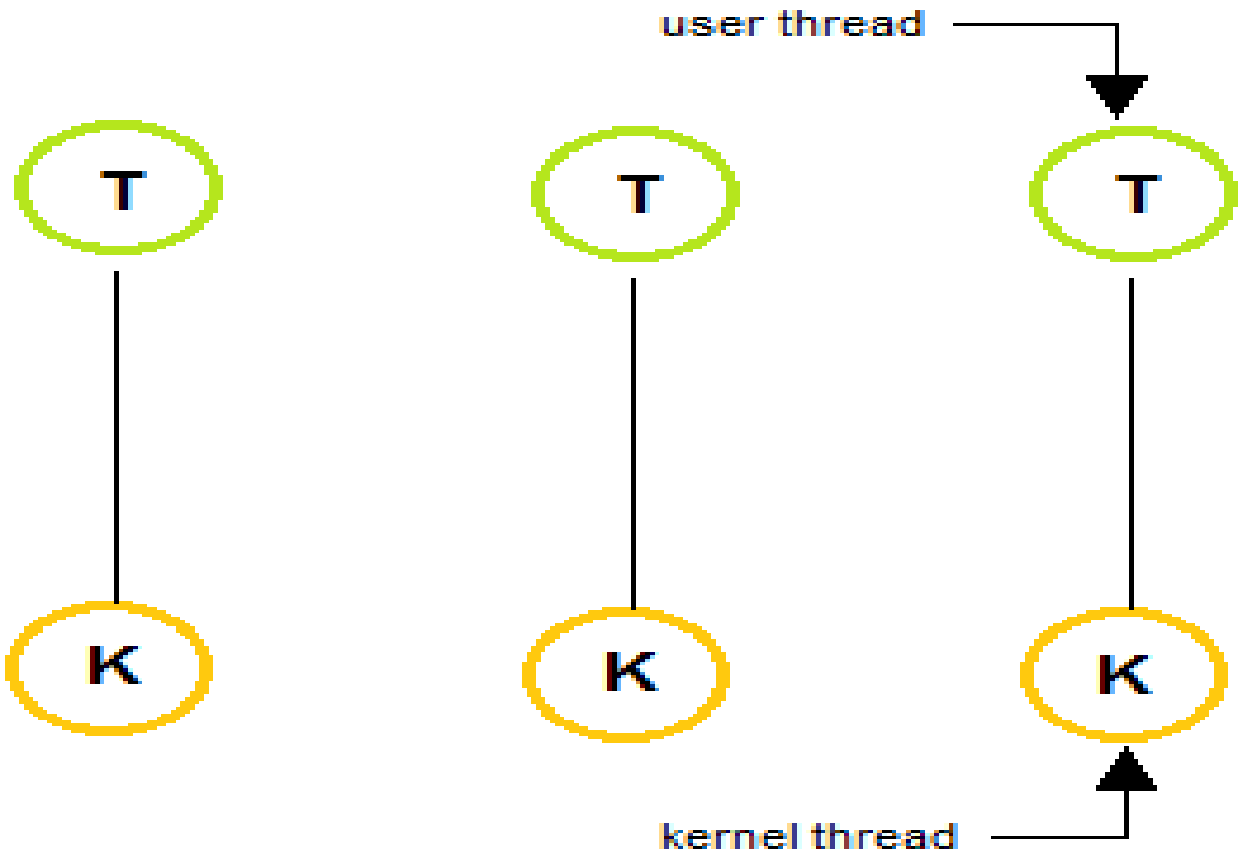
Many to One Model



One to One Model

- **The one to one model creates a separate kernel thread to handle each and every user thread.**
- **Most implementations of this model place a limit on how many threads can be created.**
- **Linux and Windows from 95 to XP implement the one-to-one model for threads.**
- **This model provides more concurrency than that of many to one Model.**

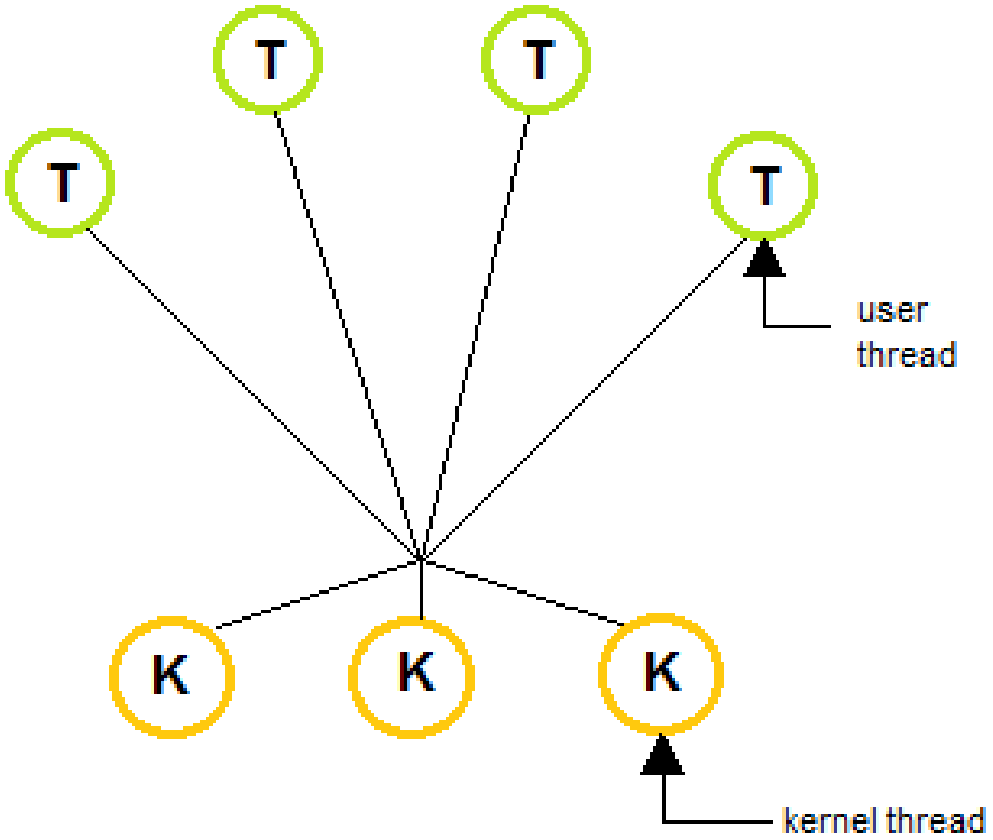
One to One Model



Many to Many Model

- **The many to many model multiplexes any number of user threads onto an equal or smaller number of kernel threads, combining the best features of the one-to-one and many-to-one models.**
- **Users can create any number of threads.**
- **Blocking the kernel system calls does not block the entire process.**
- **Processes can be split across multiple processors.**

Many to Many Model



Multithreading Issues

- **Thread Cancellation**

Thread cancellation means terminating a thread before it has finished working. There can be two approaches for this,

- **One is Asynchronous cancellation, which terminates the target thread immediately.**
- **The other is Deferred cancellation allows the target thread to periodically check if it should be cancelled.**

- **Signal Handling**

Signals are used in UNIX systems to notify a process that a particular event has occurred. Now in when a Multithreaded process receives a signal, to which thread it must be delivered? It can be delivered to all or a single thread.

Multithreading Issues

- **fork() System Call**

fork() is a system call executed in the kernel through which a process creates a copy of itself. Now the problem in the Multithreaded process is, if one thread forks, will the entire process be copied or not?

- **Security Issues**

Yes, there can be security issues because of the extensive sharing of resources between multiple threads.